전남대학교
소프트웨어중심대학사업단

바이브코딩을 위한 Git, Github 활용(기초)

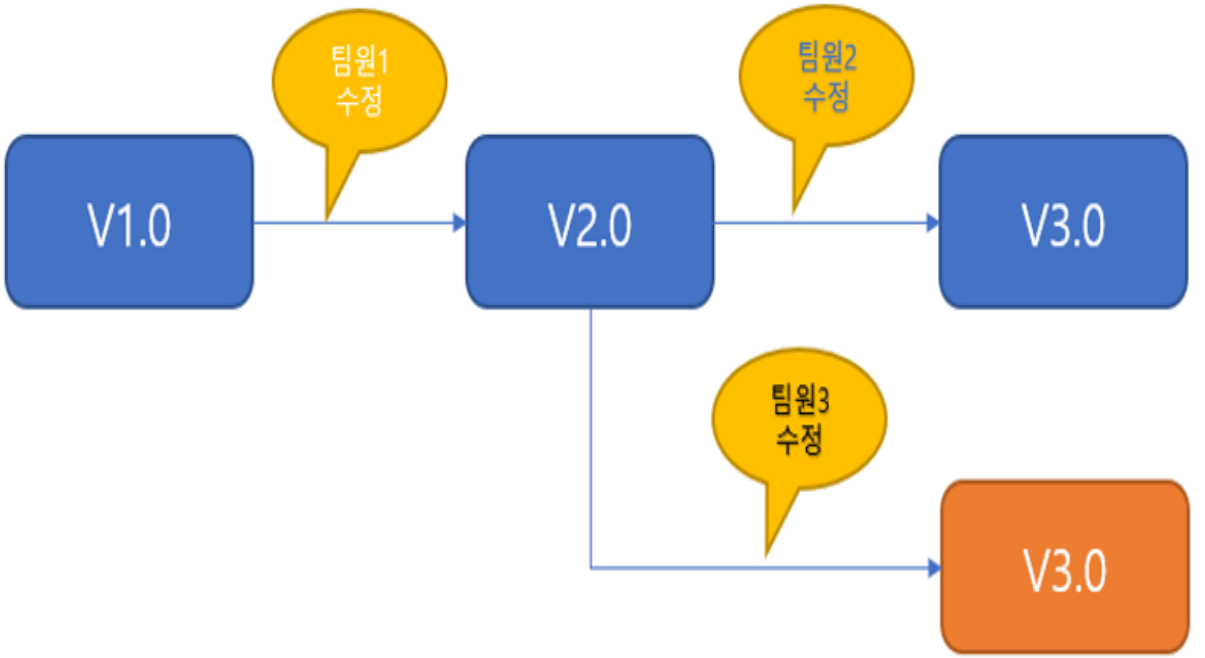
유재명(wesleyok@jnu.ac.kr)

목차

1. Git 버전관리 원리
   - 버전관리란?
   - git 설치 및 저장방식 이해
   - git 시작
2. Github 활용
3. Visual Code 연계
4. 바이브코딩 활용(구글 클라우드 + colab)

버전관리란

- 문서를 작성하다 오류가 발생할 때 `Ctrl+Z` 를 사용하는 것도 일종의 버전관리
- 문서명을 project-ver01.py, project-ver02.py 또는 project-20231017.py, project-20231017-2.py, 등 파일명을 이용하는 방법도 버전관리 형태
- 개인 뿐아니라 여러 사람이 함께 작업하는 협업 프로젝트에서는 반드시 버전관리를 해야함



버전관리란 무엇인지 글과 그림으로 간단히 요약해서 설명해 주세요

📌 1️⃣ 버전관리란 무엇인가?

버전관리(Version Control)는

👉 파일(코드, 문서 등)의 변경 이력을 저장하고 관리하는 시스템입니다.

핵심 의미

- 언제 무엇이 바뀌었는지 기록
- 이전 상태로 되돌리기 가능
- 여러 사람이 동시에 협업 가능

버전관리 형태

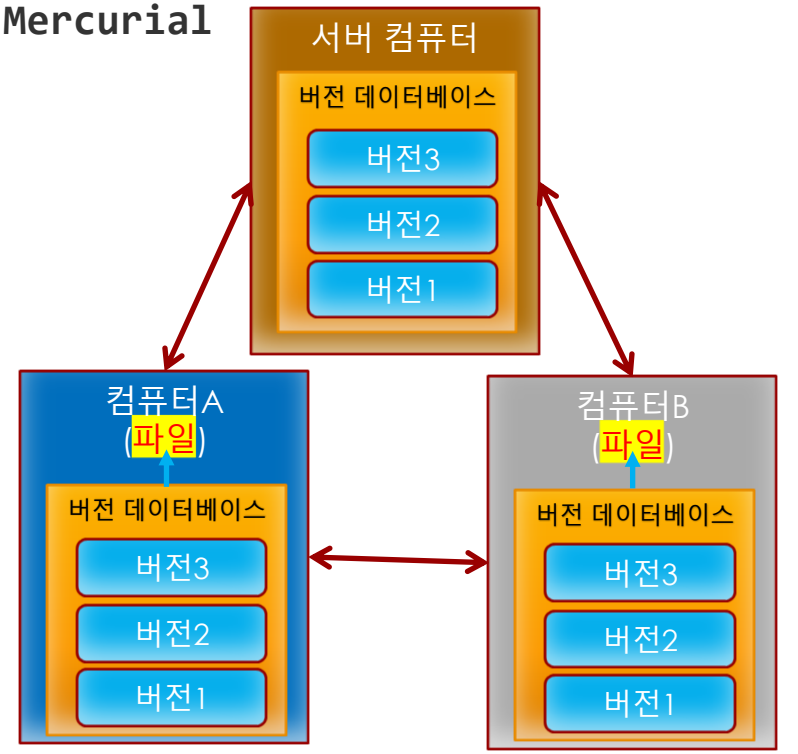
1. 클라이언트-서버 모델

- 하나의 중앙 저장소를 공유한 후 각각의 클라이언트(개발자)는 저장소의 일부분만 갖는 형태
- 중앙저장소에서 프로젝트 관리의 모든 것 처리
- 만일 서버가 고장난다면 불완전한 로컬 파일 사본들만 남게 됨
- 관련 시스템
  - CVS, CVSNT, Subversion
  - Synergy, Perforce, StarTeam등

버전관리 형태

2. 분산 모델

- 프로젝트에 참여하는 모든 클라이언트(개발자)가 전체 저장소에 대한 개별적인 로컬저장소를 갖고 작업하는 형태
- 다시 말해서, 각자가 온전한 전체 저장소의 사본을 로컬에 가지게 됨
- 관련 시스템
  - DCVS(2002), SVK(2003), Git(2005), Mercurial
  - TeamWare(1990), BitKeeper(1998) 등



Git - 버전관리시스템

- 오픈소스 linux 프로젝트는 커널 배포를 위해 1991~2002년까지 Patch와 단순 압축 파일로 관리하다 2002년부터 DVCS인 상용 BitKeeper를 사용함

> DVCS((Distributed Version Control System)

- 2005년 BitKeeper가 무료에서 유료 사용으로 전환하자 관계가 틀어졌고, 이를 계기로 Linus TOrvalds가 자체 개발함
- Git은 BitKeeper를 사용하면서 배운 교훈으로 개발 목표 세움

첫째, 빠른 속도

둘째, 단순한 구조

셋째, 비선형적인 개발(수천개의 동시 다발적인 브랜치)

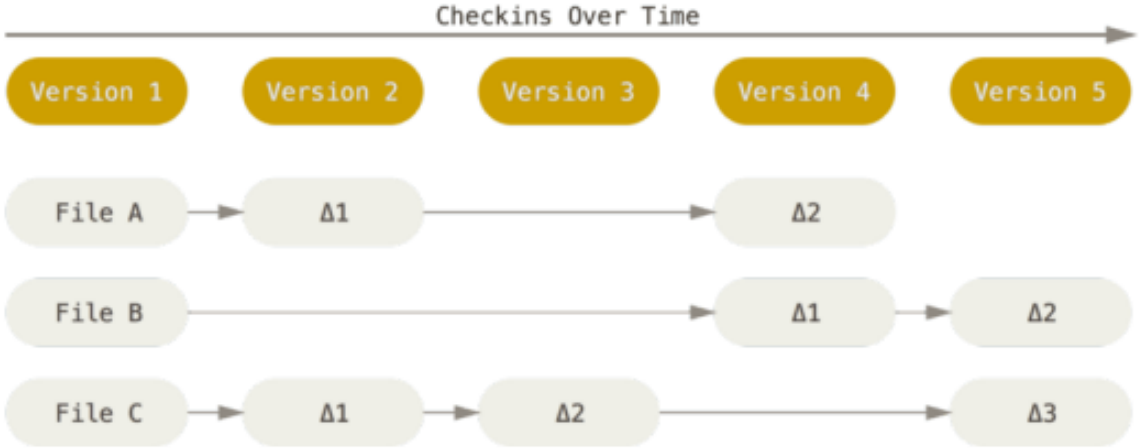
넷째, 완벽한 분산

다섯째, Linux 커널 같은 대형 프로젝트에도 유용할 것(속도나 데이터 크기면에서)

Git의 특징

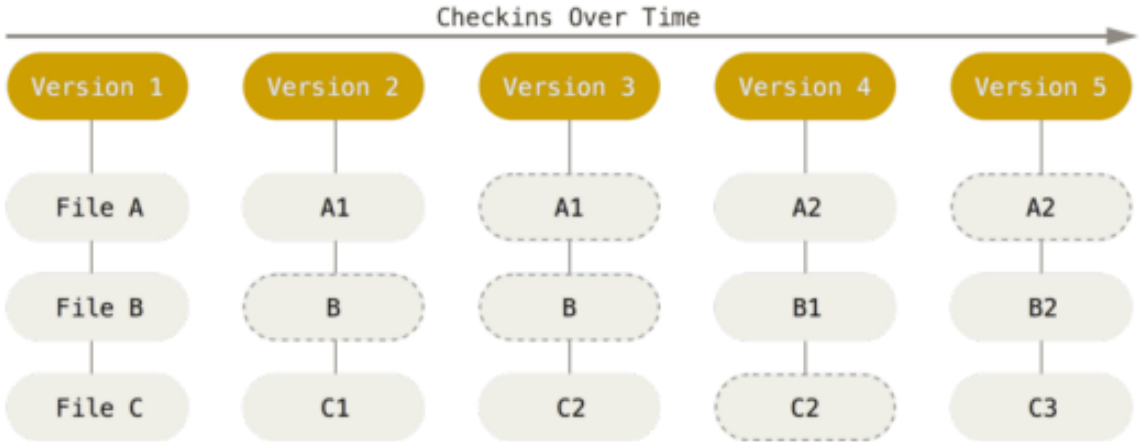
- Git의 특징은 차이가 아니라 스냅샷

  CVS, Subversion, Perforce, Bazaar 등의 VCS는 각 파일의 변화(차이)를 시간순으로 관리



차분(diff) 방식: 파일의 변경 내용을 줄 단위로 기록하여 저장하는 방식
- 변경된 부분만 저장하고
- 나머지는 참조하는 구조로
- 줄 단위로 파일 추적하기 때문에 대량의 변경 내용이나 복잡한 히스토리를 관리하는데 비교적 느림

- 그러나, Git은 시간순으로 프로젝트의 스냅샷을 저장한다.



Git의 특징

1. 스냅샷(snapshot)이란

- 증분식 저장 방식으로, 각 커밋은 스냅샷을 저장, 해당 커밋에 대해 고유한 해시(SHA-1)를 생성하며 이것으로 스냅샷을 식별하는데 사용

- Git의 내부 구조: 3가지 유형 객체저장소

① **Blob객체** : 파일 내용 자체 저장 객체, 해시값으로 구분, 동일 내용인 경우 식별되어 중복 저장 방지, 즉 파일이 여러 브랜치나 커밋에서 동일하게 존재하더라도 Git은 한번만 저장되며, 한 번 생성되면 변경되지 않으며, 압축된 형태로 .git/objects에 저장됨

- 목록보기: git ls-tree HEAD 또는 체크섬,
- 내용보기: git cat-file -p 체크섬
- 타입보기: git cat-file -t 체크섬 (타입종류: blob, tree, commit)

② **Tree객체**: 디렉토리와 파일목록 관리 객체, 각 커밋의 스냅샷은 트리 객체로 파일 및 하위 디렉토리 포인터

③ **커밋객체**: 부모 커밋에 대한 포인터와 트리 객체를 가리키는 포인터

- 원본 데이터 종속성

- 원본 데이터가 스토리지 내에 저장되기 때문에 하드웨어 벤더 변경시 스냅샷 사용 불가

- 전체 데이터를 복제하지 않기 때문에, 변경된 파일에 대한 스냅샷만 저장하기 때문에 저장 공간을 크게 절약 및 복원 속도 빠름

- 스냅샷 저장 과정은 효율성을 높이기 위해 zlib 압축과 중복 제거 기능 사용

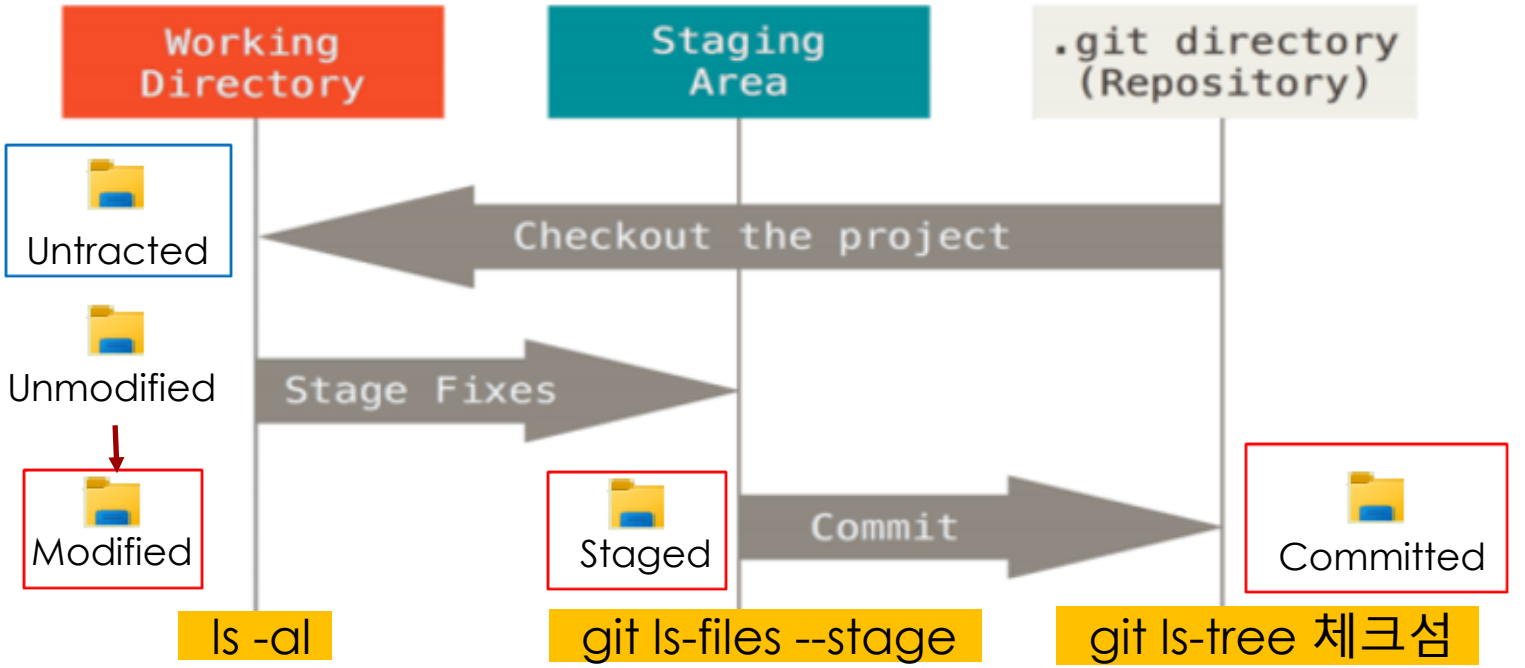
- 빈번한 업데이트/수정이 있는 작업시 더 효과적

Git의 특징

Committed: 데이터가 로컬 데이터베이스에 안전하게 저장되었다는 뜻

Modified: 수정한 파일을 아직 로컬 데이터베이스에 커밋하지 않는 상태

Staged: 현재 수정한 파일을 곧 커밋할 것이라고 표시한 상태



- Git 디렉토리(.git) : Git이 프로젝트의 메타데이터와 객체 데이터베이스를 저장하는 곳
  다른 컴퓨터에 있는 저장소를 Clone할 때 Git 디렉토리가 생성
- Git 기본 프로세스
  1. 워킹 트리에서 파일을 수정한다.
  2. Staging Area에 파일을 Stage해서 커밋할 스냅샷을 만든다.
  3. Staging Area에 있는 파일들을 커밋해서 Git 디렉토리에 영구적인 스냅샷으로 저장한다.

개발환경 구축

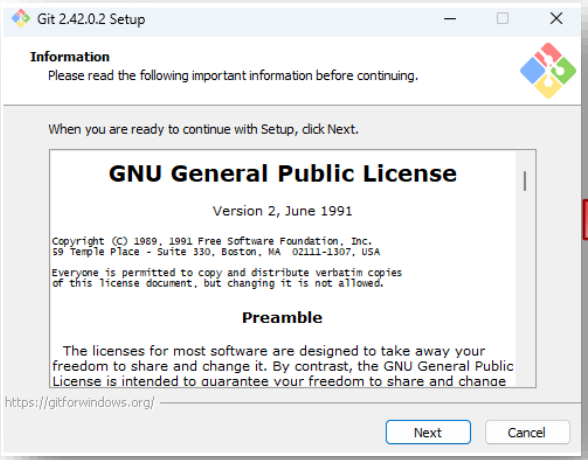
1. 윈도우에 Git 설치

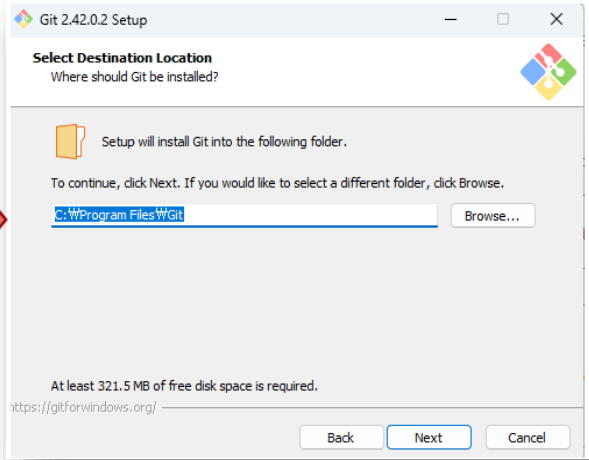
- https://git-scm.com/downloads

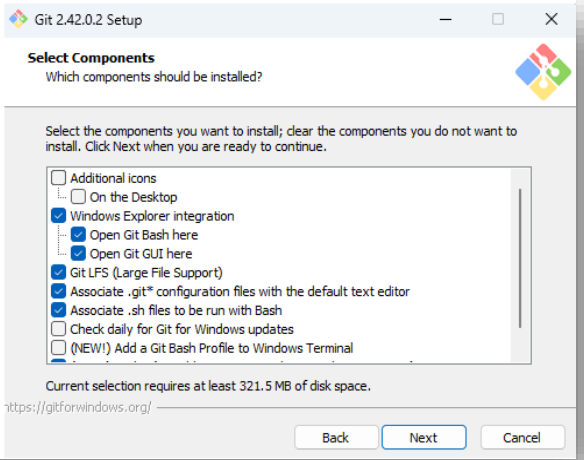
2. 리눅스 등에 Git 설치

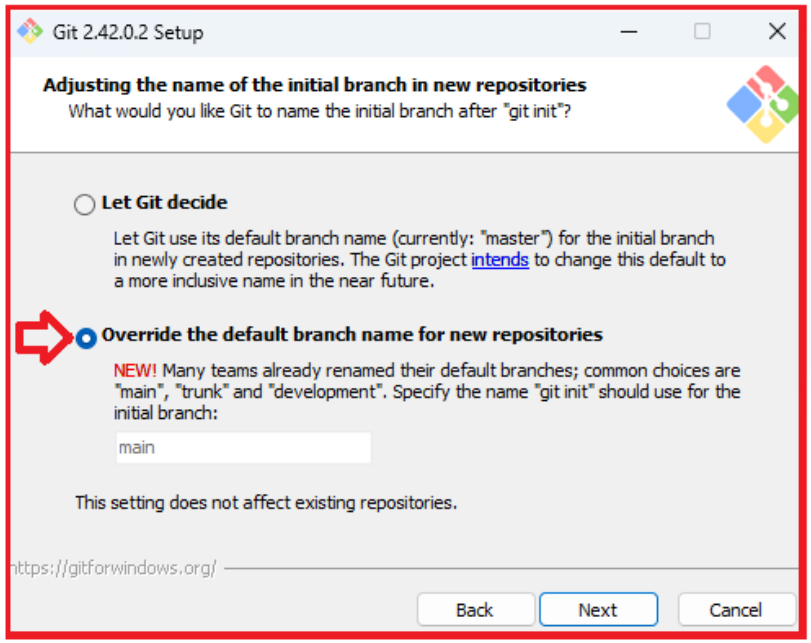
- Fedora 또는 Centos 등 RPM기반 패키지 시스템
  - sudo dnf install git-all
- Ubuntu등 데비안 계열
  - sudo apt install git-all
- Mac OS
  - brew install git

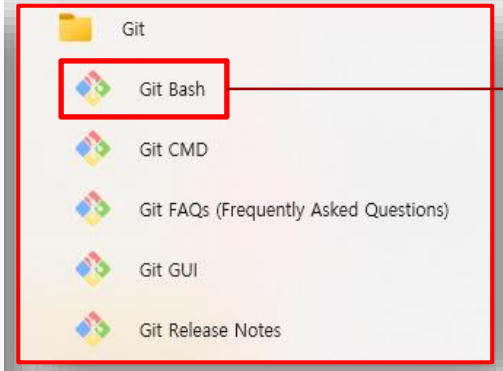
2. 실습환경 구축: GIT

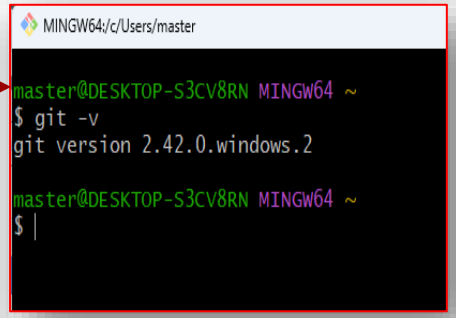

Git 2.42.0.2 Setup
Information
Please read the following important information before continuing.
When you are ready to continue with Setup, click Next.
GNU General Public License
Version 2, June 1991
Copyright (C) 1989, 1991 Free Software Foundation, Inc.
59 Temple Place - Suite 330, Boston, MA 02111-1307, USA
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.
Preamble
The licenses for most software are designed to take away your
freedom to share and change it. By contrast, the GNU General Public
License is intended to guarantee your freedom to share and change
https://gitforwindows.org/
Next
Cancel


Git 2.42.0.2 Setup
Select Destination Location
Where should Git be installed?
Setup will install Git into the following folder.
To continue, click Next. If you would like to select a different folder, click Browse.
C:₩Program Files₩Git
Browse...
At least 321.5 MB of free disk space is required.
https://gitforwindows.org/
Back
Next
Cancel


Git 2.42.0.2 Setup
Select Components
Which components should be installed?
Select the components you want to install; clear the components you do not want to install. Click Next when you are ready to continue.
Additional icons
On the Desktop
Windows Explorer integration
Open Git Bash here
Open Git GUI here
Git LFS (Large File Support)
Associate .git* configuration files with the default text editor
Associate .sh files to be run with Bash
Check daily for Git for Windows updates
(NEW!) Add a Git Bash Profile to Windows Terminal
Current selection requires at least 321.5 MB of disk space.
https://gitforwindows.org/
Back
Next
Cancel


Git 2.42.0.2 Setup
Adjusting the name of the initial branch in new repositories
What would you like Git to name the initial branch after "git init"?
Let Git decide
Let Git use its default branch name (currently: "master") for the initial branch in newly created repositories. The Git project intends to change this default to a more inclusive name in the near future.
Override the default branch name for new repositories
NEW! Many teams already renamed their default branches; common choices are "main", "trunk" and "development". Specify the name "git init" should use for the initial branch:
main
This setting does not affect existing repositories.
https://gitforwindows.org/
Back
Next
Cancel


Git
Git Bash
Git CMD
Git FAQs (Frequently Asked Questions)
Git GUI
Git Release Notes


MINGW64:/c/Users/master
master@DESKTOP-S3CV8RN MINGW64 ~
$ git -v
git version 2.42.0.windows.2
master@DESKTOP-S3CV8RN MINGW64 ~
$

✦ Git 필수 명령어

■ 환경 구성 방식

- 환경 구성은 로컬 > 글로벌 > 시스템 순으로 저장소에 설정항목이 우선 적용

[설정]
git config --global user.name "[사용자이름]"
git config --global user.email "[이메일계정]"
[줄바꿈 에러메시지 감추기]
git config --global core.autocrlf true

[한글 파일,폴더 문제 처리]
git config --global core.precomposeunicode false
git config --local core.quotepath false
[기본 브런치 설정]
git config --global init.defaultbranch main

[보기]
git config --system --list : system 환경설정 보기
git config --global --list : global 환경설정 보기
git config --local --list : local 환경설정 보기
git config --list : 전체 환경설정 보기
git config --system --get credential.helper 인증 방식 보기

[삭제]
git config --unset user.name : 사용자이름 지우기
git config --unset user.email : 이메일계정 지우기

1) Git 환경 구성(2)

✦ 한글(파일, 폴더) 다루기

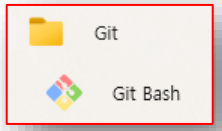
■ 파일 또는 폴더가 한글일때

```
브랜치가 'origin/master'에 맞게 업데이트된 상태입니다.

추적하지 않는 파일:
  (커밋할 사항에 포함하려면 "git add <파일>..."을 사용하십시오)
        "Programmers_Java/\354\231\204\354\240\204\355\203\220\354\203\211/"

커밋할 사항을 추가하지 않았지만 추적하지 않는 파일이 있습니다 (추적하려면 "git
add"를 사용하십시오)
```

- 위와 같이 무슨 내용인지 확인도 어렵고, 불편하다
- Git 환경의 초기설정 인코딩 문제
- 해결방안
  - 로컬영역 환경구성: git config --local core.precomposeunicode false
  - 글로벌영역 환경구성: git config --global core.quotepath false

```
> git config --local core.precomposeunicode false
> git config --global core.quotepath false
```

2. 실습환경 구축 : 간단한 Git 명령어 사용




```
$ mkdir mywork
$ cd mywork
$ git init
$ git branch -M main      <= master를 main브런치로 변경
master@DESKTOP MINGW64 mywork (main)
```

$ git config --global init.defaultbranch main

전체 환경 설정 상태 보기

```
$ git config --list
master@DESKTOP MINGW64 mywork (main)
```

빠져나오는 방법: Ctrl+Z 후 "Q"

```
$ git config --get user.email
Your-mail@mail
$ git config --get user.name
Your name

$ git config --unset user.name      <= 설정 해지
```

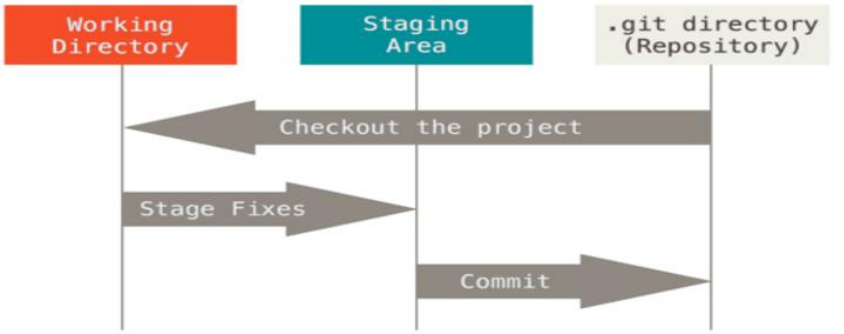
[개인정보 등록]

```
$ git config –local user.name "yoojaemyeong"
$ git config –local user.email "yoojaemyeong@gmail.com"
```

[로컬 실습] - 작업내용

✦ Git 시작 : 새 폴더 생성(mywork)

- git init



✦ 작업일지(python, 한글cprogram)

- 1일차
  - notepad workflow.txt 작업일지 시작 > git add workflow.txt > git commit –m “1. 작업시작”
- 2일차
  - notepad add.py sub.py > git add add.py sub.py > git commit –m “2. Add,Sub함수 추가”
- 3일차
  - notepad multiply.py > git add multiply.py > git commit –m “3. Multiply함수 추가”
- 4일차
  - notepad main.py > git add main.py > git commit –m “4. 메인프로그램 추가”
- 5일차
  - notepad workflow.txt > git commit –am “5. Python프로젝트 완료 "

[로컬 실습] - 작업 검토

전남대학교 소프트웨어중심대학사업단

- 로그 보기
  - git log --oneline --all --decorate --graph
  - git status 현재 상태
  - git show [체크섬]
- 이동하기
  - git checkout [체크섬] 체크섬으로 이동하여 각 날짜별 작업내용 보기
  - git checkout HEAD~3 체크섬을 3단계 이전으로 HEAD 이동
  - git checkout main 원위치(HEAD->main)
  - git ls-files –stage 버전 관리 대상 파일 보기
  - git ls-tree [체크섬] 또는 HEAD^^^ 체크섬 트리 구조 보기
  - git cat-file –t 또는 -p [체크섬] -t(type: : blob, tree, commit) , -p 내용보기
- 작업폴더 > Staging > Repository 상태 변화
  - ls 또는 dir : 작업폴더의 목록 보기
  - git add 파일명 또는 폴더명 : 작업폴더 > Staging Area로 추적 대상 등록
  - git commit –m " 메시지 " : Staging Area에서 로컬저장소로 저장
- 파일 내용 비교
  - git diff : Unstaged된 파일들의 내용 비교
  - git diff –staged : Staging 파일 간의 변경 사항 출력
  - git diff [체크섬1] [체크섬2] : 두 커밋 사이의 파일 간의 변경 사항 출력

[로컬 실습] - 브런치 작업

전남대학교 소프트웨어중심대학사업단

✦ 작업순서

git checkout [체크섬] mywork폴더에서 1일차 [체크섬]으로 이동

git checkout -b cprogram 새로운 브런치 생성

Cprogram 브런치 작업 시작

[작업내용] 첫째날

① git log --oneline -all 로그 보기

② 작업 추가(add.hwp) : int add(int a, int b){ … }

③ git add add.hwp > git commit -m “1. 한글 add함수 추가"

[작업내용 추가] 둘째날

① 작업 추가(multiply.hwp) : int multiply(int a, int b){ … }

② git add multiply.hwp > git commit -m "2. 한글 multiply함수 추가"

[작업내용 추가] 셋째날

① 작업 추가(mod.hwp) : int mud(int a, int b){ … }

② git add mod.hwp > git commit -m "3. 한글 mod함수 추가"

③ 로그 보기(git log --oneline --all)

[작업내용 추가] 넷째날

① 작업 내용 추가(workflow.txt에 작업 내용 추가)

② git commit -am “1. workflow.txt 작업일지 정리"

❖ 브런치로 이동하면서 저장상태 확인

[로컬 실습] - 병합

✦ 작업순서

Main 브런치로 이동 및 브런치 병합

- [작업내용]

① 로그 보기(git log --oneline –all --decorate --graph)

② Main브런치로 이동(git switch main)

③ 병합(git merge cprogram")

④ 로그 보기(git log --oneline –all --decorate --graph)

정리

- 로그 보기
  - git log --oneline --all --decorate --graph
  - git status 현재 상태
  - git show [체크섬]
- 이동하기
  - git checkout [체크섬] 체크섬으로 이동하여 각 날짜별 작업내용 보기
  - git checkout HEAD~3 해당 체크섬으로 HEAD 이동
  - git ls-files –stage 버전 관리 대상 파일 보기
  - git ls-tree [체크섬] 또는 HEAD^^^ 체크섬 트리 구조 보기
  - git cat-file –t 또는 -p [체크섬] -t(type: : blob, tree, commit) , -p 내용보기
- 작업폴더 > Staging > Repository 상태 변화
  - ls 또는 dir : 작업폴더의 목록 보기
  - git add 파일명 또는 폴더명 : 작업폴더 > Staging Area로 추적 대상
  - git commit –m " 메시지 " : Staging Area에서 로컬저장소로 저장
- 브런치 생성 및 병합
  - git branch : 현재 작업중인 브런치 이름
  - git branch [새브런치이름] 또는 git checkout –b [새브런치이름] : 새 브런치 생성
  - git switch [브런치이름] : 브런치이름으로 HEAD 이동
  - git checkout [체크섬] : 체크섬으로 HEAD 이동
  - git merge [새브런치] : 현재 작업중인 브런치에 새브런치를 병합

그외 중요 명령어

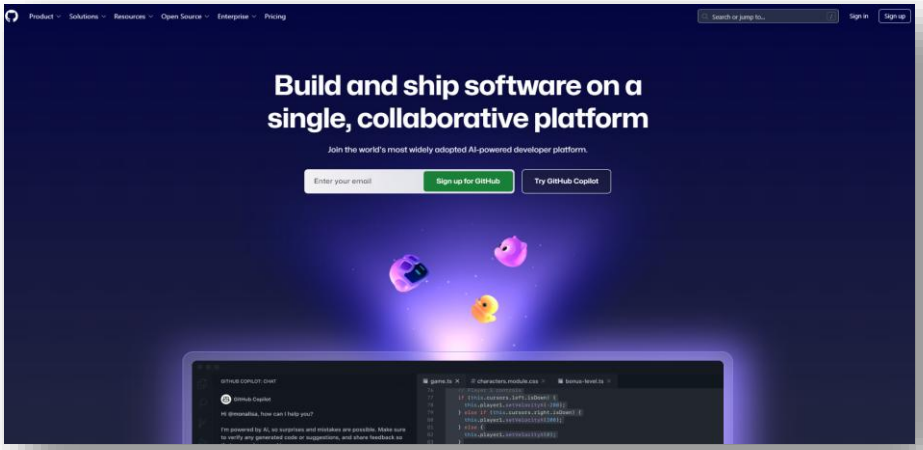
- git stash 인덱스 영역에 트래킹되는 파일을 임시 영역으로 저장하고, modified 부분을 Working Directory에서 제거, 기본 명칭 WIP로 저장됨
  - git stash save [저장명] -um "메시지"
  - git stash list
  - git apply [stash 인덱스]
  - git stash drop : 가장 최신 stash 제거
  - git stash drop [stash 인덱스]
  - git stash pop : 최근 작업 불러오고, stash 영역에서 삭제
  - git stash clear : stash 영역 모든 내용 삭제

- git blame [파일명] : 파일 작성자 정보 확인
  - git blame [커밋해시] [파일명]
  - git blame –L [시작라인],[끝라인] [파일명]
  - git blame –e [파일명] : 작성자 이름 대신 이메일 정보 표시
  - git blame –s [파일명] : 커밋 해시만 표시

2. Github 활용

Github 개요

- Github 서비스: https://github.com/



- Github란?
  - Git 저장소를 온라인에서 관리하는 서비스
  - 협업, 이슈관리, Pull Request, 백업
  - Repository: 프로젝트 단위 관리 공간

- 예시
  - 원격저장소: Github → New Repository → 이름: python-exam
  - 파일: hello.py

```
def hello():
  print("hello")
if __name__ == "main":
  hello()
```

  - 로컬에서 Github연결

```
git remote add origin https://github.com/username/python-exam.git
git branch -M main
git add hello.py
git commit -m "파이썬 hello"
git push origin main
```

Github 회원가입

실습 (1) GITHUB 사이트에서 신규 계정 생성하기 (2) 기존 계정이 있는 사람은 Gmail+ 계정 생성

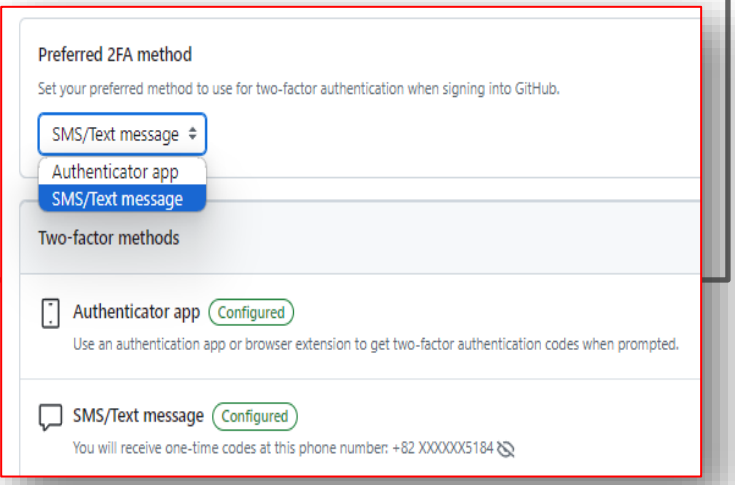
1. Github 계정 생성

2. 팀개발 테스트를 위해 **Gmail +** 기능을 사용
   - 예로, yoojaemyeong@gmail.com 계정이 있다면
   - yoojaemyeong+one@gmail.com, yoojaemyeong+two@gmail.com 등 생성 가능 모두 기본 메일 계정으로 수신됨

yoojaemyeong+3@gmail.com

3. 계정 생성 후 반드시 기록하여 보관할 것

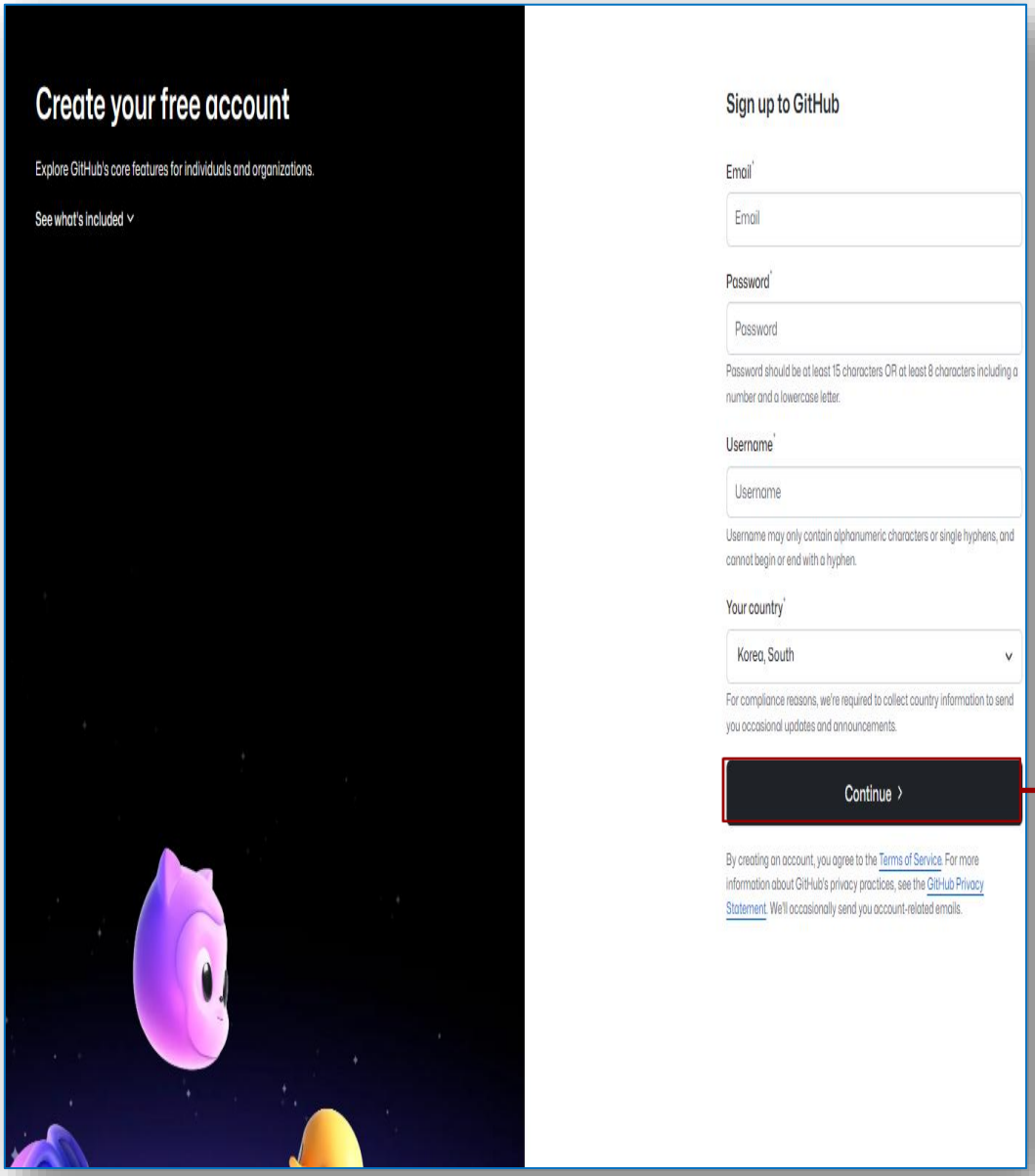
4. 추가적으로 Two-factor authentication 등록
   - [Github로그인] > [Settings] > [Password and authentication]

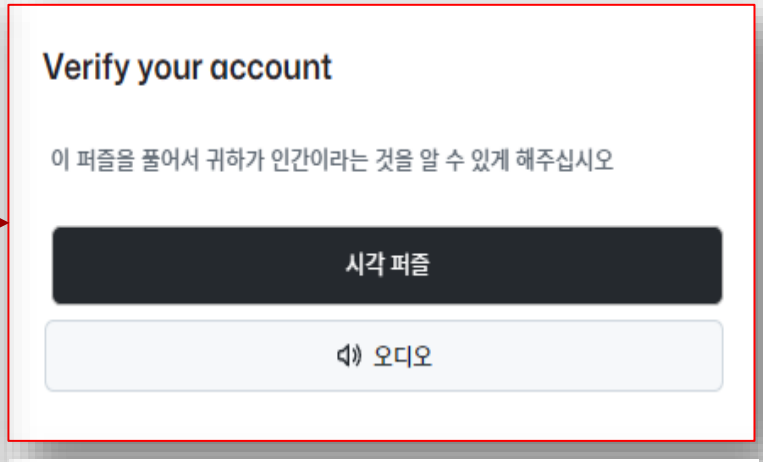


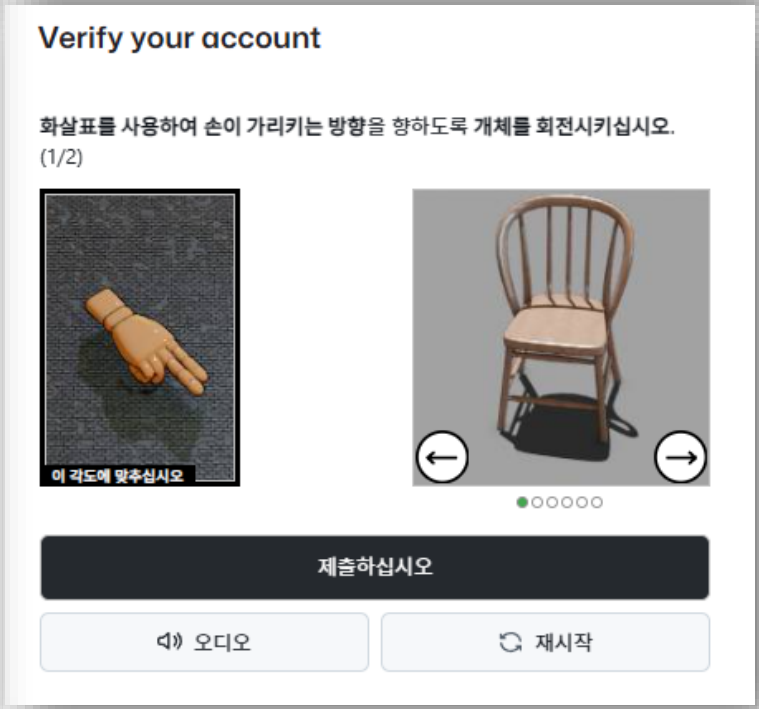
Preferred 2FA method : 웹사이트 로그인시 이 앱을 활용하여 일회용 비밀번호 생성

1. [iOS ,Android] 용 2단계 인증 클라이언트 모바일 앱 [Microsoft Authenticator] 설치하여 활용
2. 또는 [SMS/Text message] 활용

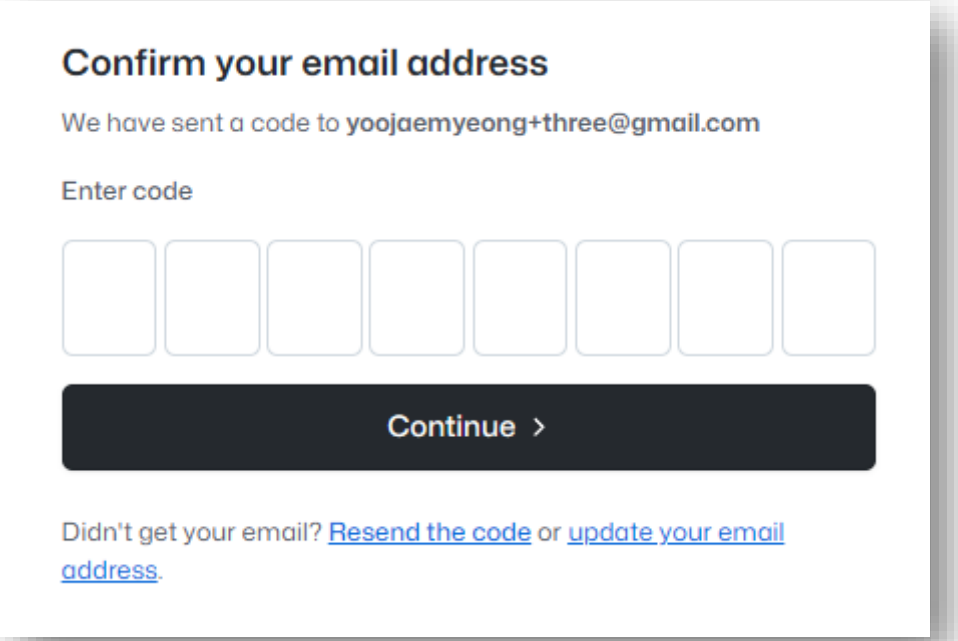
Github 회원가입(1)



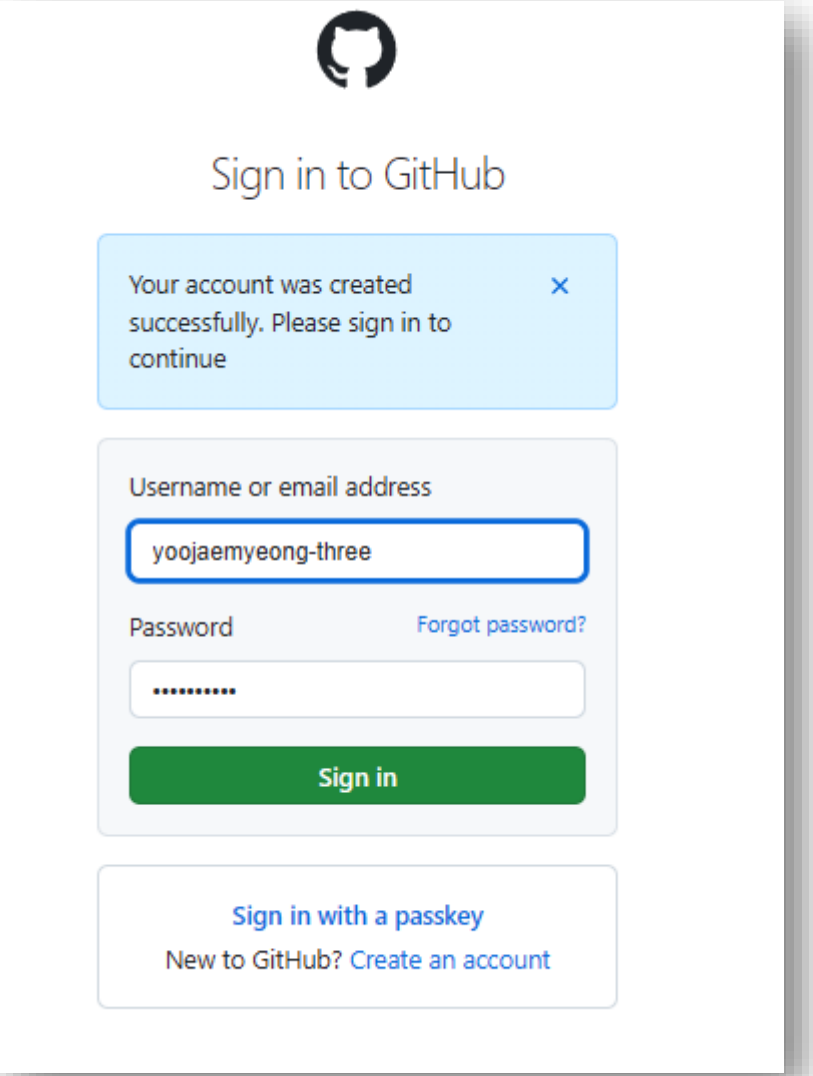




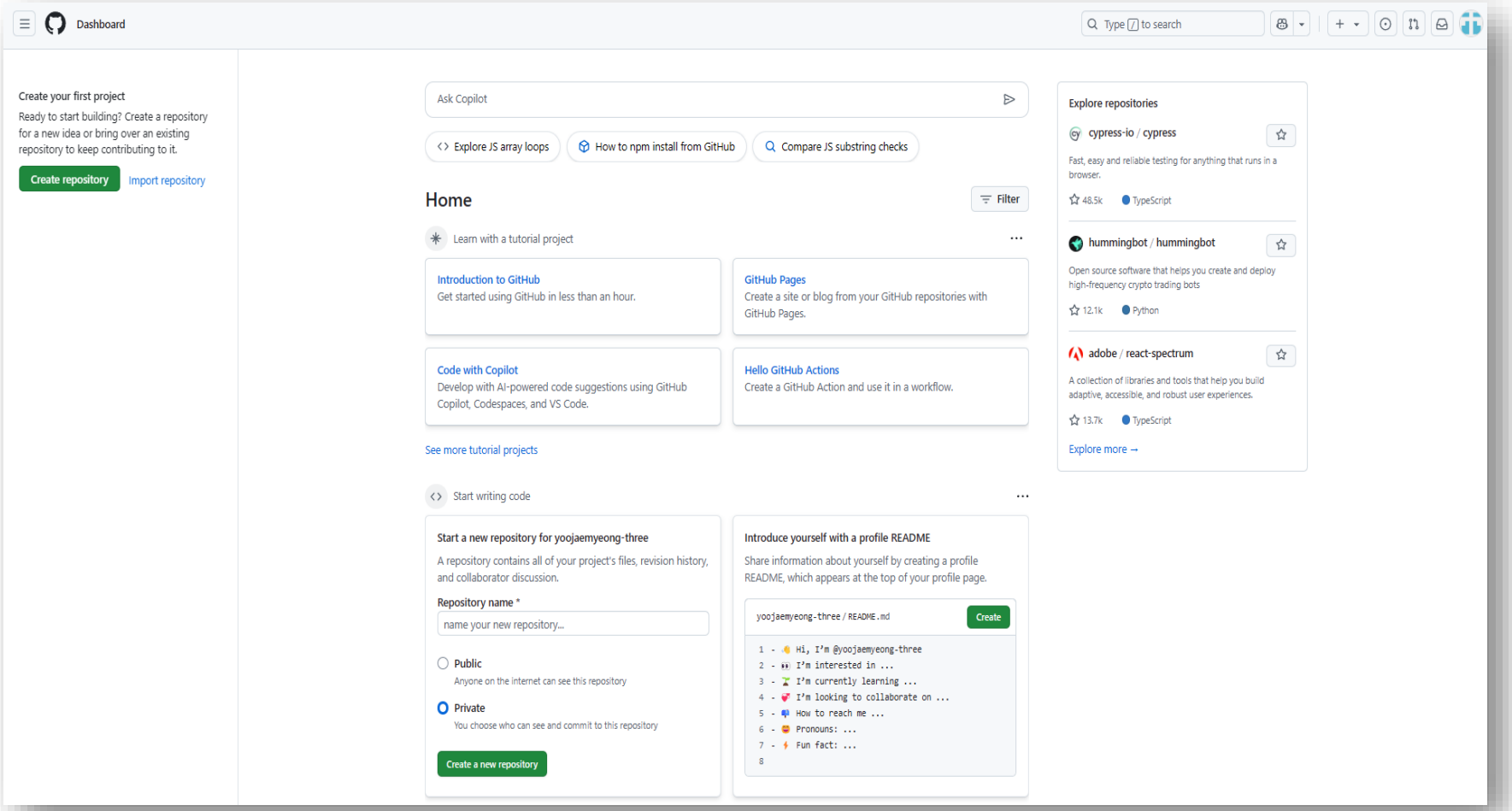
Github 회원가입(2)



본인이 등록한 이메일 답신 확인하여
등록번호 입력



Github 회원가입(3)

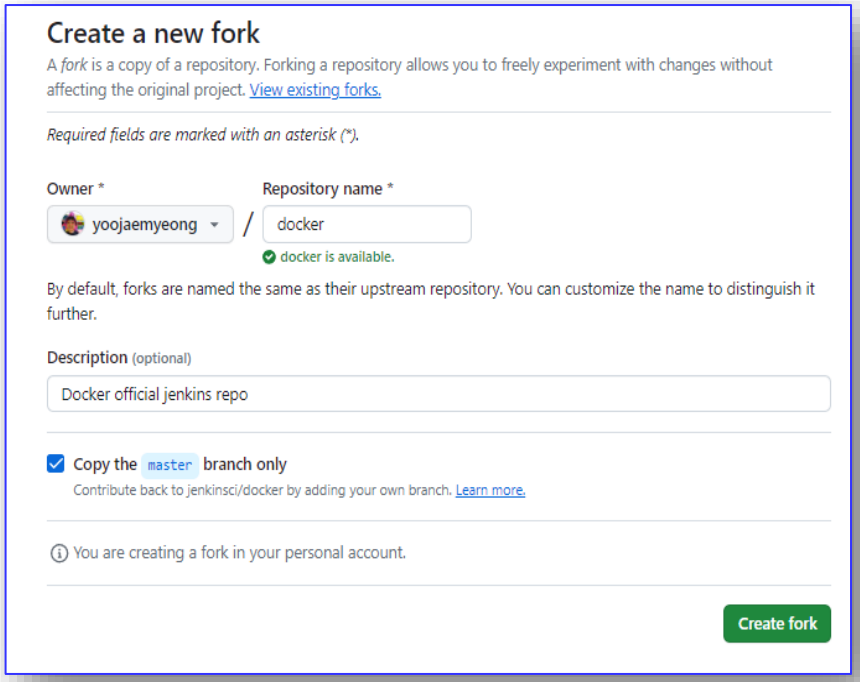


Github 핵심기능

- Github Repository
  - Code, Issues, Wiki, Insights, Settings 등 코드 버전 관리 저장소
- Github Packages
  - Javascript(npm), Python(pip), Java(pom, gradle), Nuget 등 저장소 기능 수행
- Github Actions
  - github저장소에서 개발 워크플로[빌드, 테스트, 배포] 수행(CI/CD)
- Github Fork
  - 공개 소스를 사용자의 저장소로 분리하여 개발 진행
- Github Pull requests : 공동 작업
- Github Projects
  - Issues, Pull requests와 연계되어 필터링, 정렬, 그룹화 등 작업 보드 및 로드맵

Github 핵심기능 : Fork

- Github에 로그인
- 찾기 : docker
- jenkinsci / docker 선택 후 Fork 클릭
- 내용 확인 후 “Create fork” 클릭



Github 핵심기능 : Packages

- Public packages
  - Free
  - 무제한
- Private packages
  - 유료, 500MB / pro2GB / team2GB / enterprise50GB

Github 핵심기능 : Repository

Repositories 1 → New

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

Owner * yoojaemyeong3 / Repository name * project1

Great repository names are short and memorable. Need inspiration? How about miniature-sniffle ?

Description (optional) 원격저장소 만들기

- Public — Anyone on the internet can see this repository. You choose who can commit.
- Private — You choose who can see and commit to this repository.

Initialize this repository with:

- [x] Add a README file — This is where you can write a long description for your project. Learn more about READMEs.

Add .gitignore: .gitignore template: None

Choose which files not to track from a list of templates. Learn more about ignoring files.

Choose a license: License: None

A license tells others what they can and can't do with your code. Learn more about licenses.

This will set `main` as the default branch. Change the default name in your settings.

You are creating a public repository in your personal account.

Create repository

원격저장소 생성

project1

Pin | Unwatch 0

main | 1 Branch | 0 Tags | Go to file | Add file | Code

yoojaemyeong3 Initial commit 650c49e · now 1 Commit

README.md Initial commit now

README

project1

원격저장소 만들기

자격 증명 관리(토큰)

- 인증(credential.helper)
  - manager(기본)
  - Cache
  - Store

- Manager: 윈도우 운영체제에서 제공하는 자격증명관리자에 사용자 인증 정보를 저장

- Cache:
  - git config credential.helper “cache --timeout=30”
  - 지정한 시간(30초)동안 ID와 패스워드 저장

- Store:
  - git config credential.helper “store --file ~/.git-credentials”
  - 파일을 이용하여 ID와 패스워드를 저장하는 방식(무제한)

- 인증 지우기
  - “manager” 자격 증명인 경우, 공용PC 실습 후에는
    윈도우 자격증명 관리자를 실행하여 git:으로 시작하는 자격 증명 삭제

자격 증명 관리(토큰)

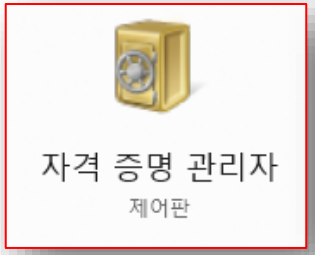
- 자격 증명 상태 보기 명령어

```
git config credential.helper
manager
git config --global credential.helper
git config --local credential.helper
git config --system credential.helper
```

- 자격증명이 필요한 이유: 서로 다른 시스템을 연결하기 위해 필요, 여기서는 Github 원격저장소 사용 목적

- **윈도우 자격증명(credential) 관리**
  - **credential.helper 값이 manager이면**
  - **윈도우에서 제공하는 자격증명관리자에 저장**

[시작]->[검색창]에 "자격" 입력 후 "자격 증명 관리자"실행



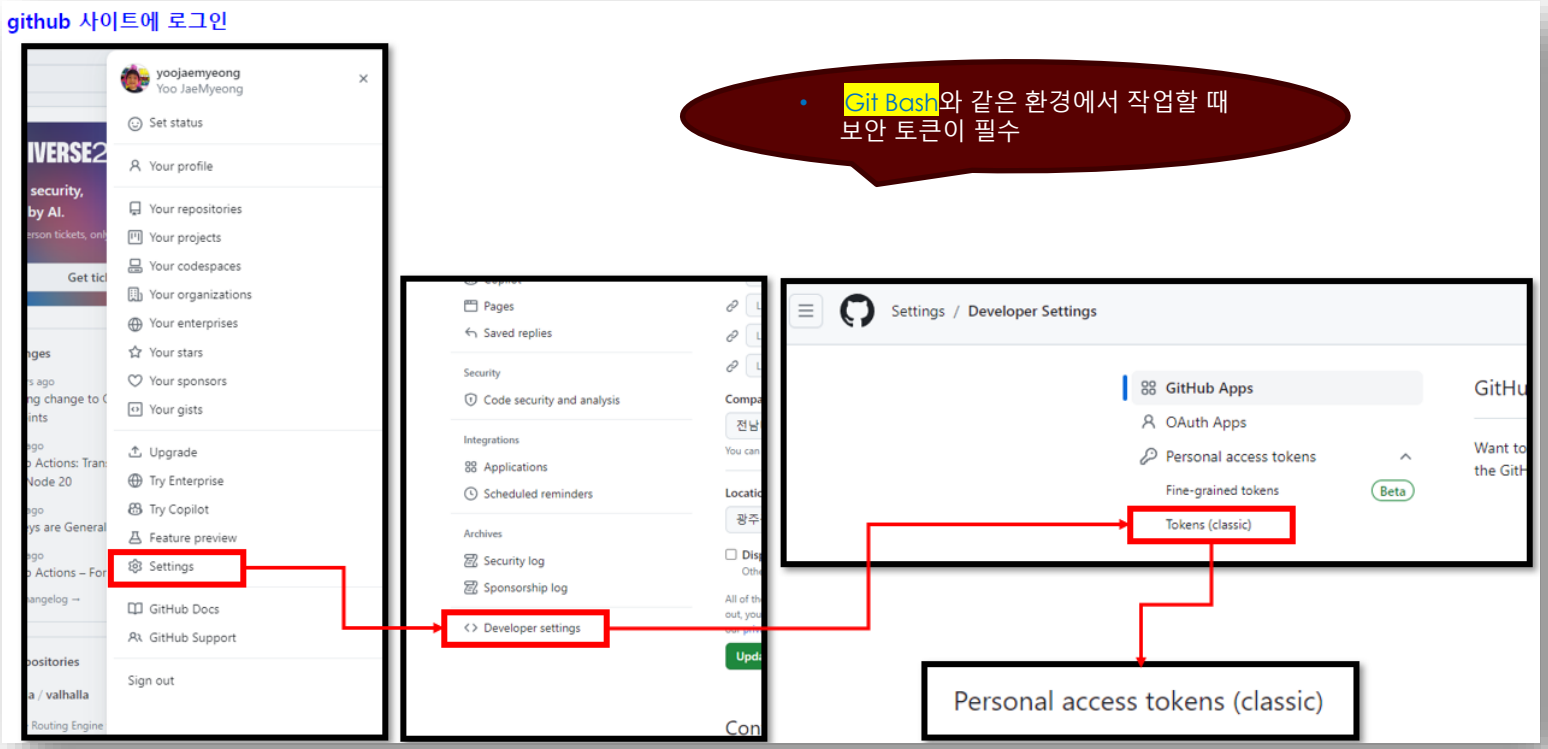
계정 정보가 없는 상태에서 `git push` 실행하면 로그인창이 떠서 ID/패스워드를 묻는다.
계정이 있다면 아래와 같은 정보를 볼 수 있다.

```
$ git push
Everything up-to-date
```

✔ 맥OS에서 인증관련 명령어는 `osxkeychain`

자격 증명 관리(토큰)

실습 (1) GITHUB 사이트에서 본인 계정으로 로그인 후 아래 내용 참조하여 토큰 생성하기

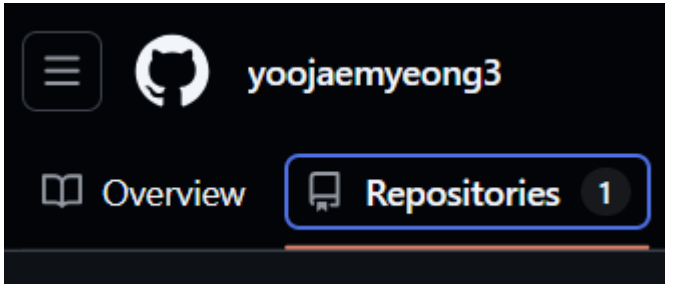


- Git Bash와 같은 환경에서 작업할 때 보안 토큰이 필수

토큰이름(Note) : ID
토큰 : 패스워드

[Note] : my-first-access-token
[Expiration] : 2일
일단 모든 박스 체크 후 [Generate token] 클릭
[Note] 및 [토큰]을 안전한 곳(파일)에 저장하기

자격증명(git clone: 원격->로컬)



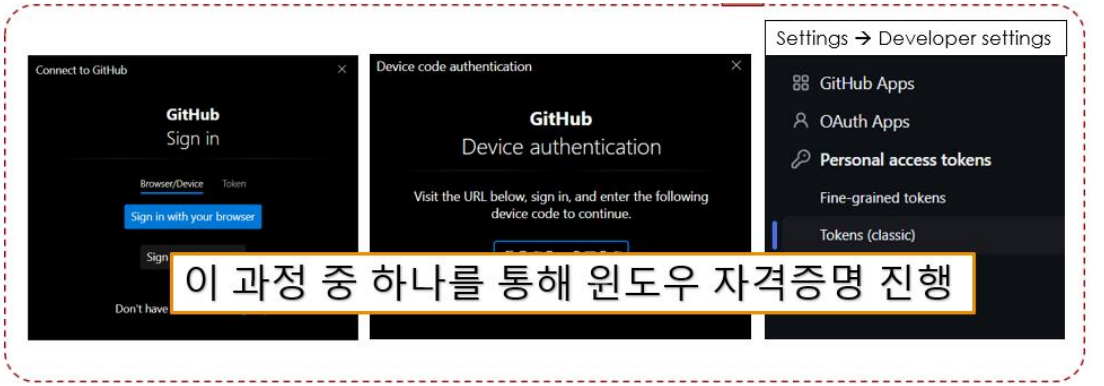
Github 계정 생성 및
원격저장소 생성

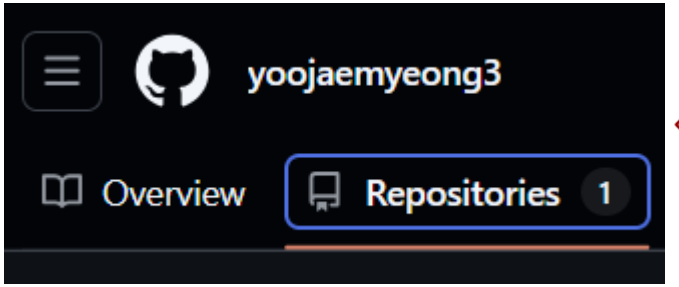
https://github.com/[userid]/project1.git

내 컴퓨터에서 git clone 활용

```
workspace>git clone https://github.com/[userid]/project1.git
```

자격증명(git pull: 로컬->원격)





Github 계정 생성 및 원격저장소 생성

https://github.com/[userid]/project1.git

내 컴퓨터에서 git pull origin main 활용

```
git init
git config –global user.name "이름"
git config –global user.email "git등록메일주소"
git remote add origin [github원격저장소주소]
git remote –v
git pull origin main
```

```
PS F:\Projects\git_workplace\first> git pull origin main
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 871 bytes | 1024 bytes/s, done.
From https://github.com/yoojaemyeong3/first
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
PS F:\Projects\git_workplace\first>
```

Github 자격 증명 관리(SSH)활용 버전관리 실습1

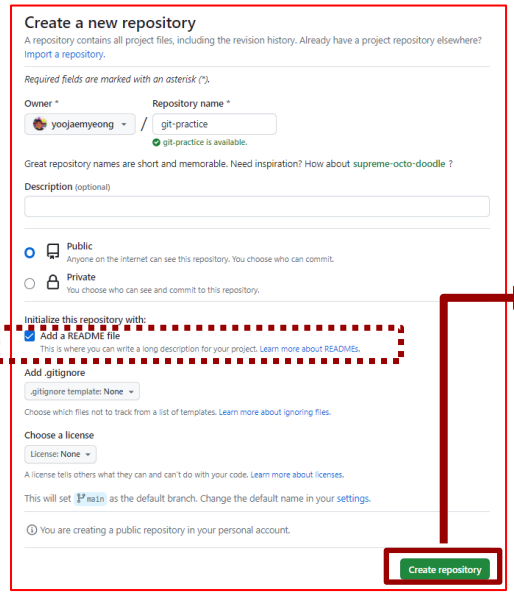
실습

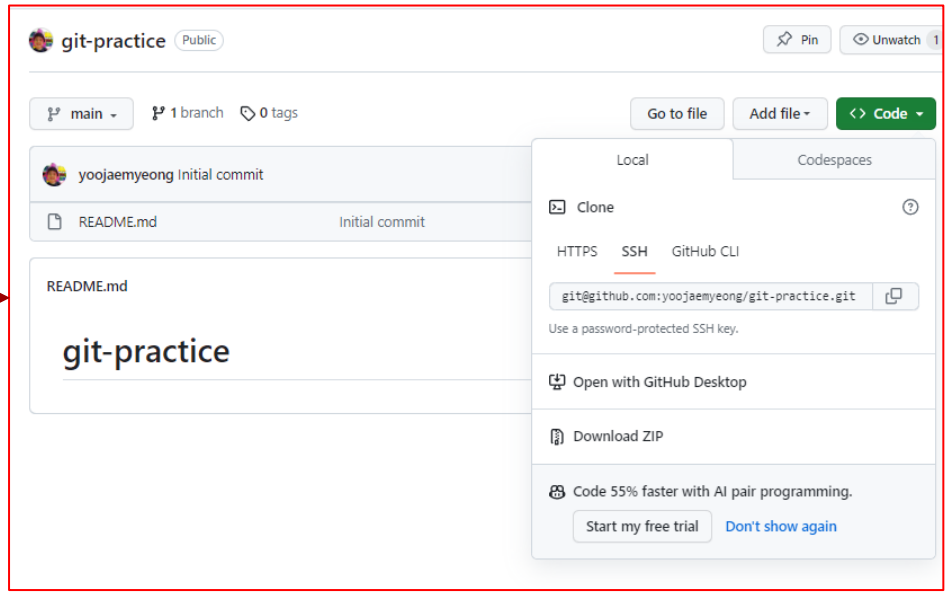
(1) GITHUB 사이트에서 본인 계정으로 로그인 후 “git-practice”이름의 원격저장소 생성하기

1. 원격저장소 생성하기

- github에 로그인 후 git-practice Repository 생성
- Add a README file 체크 후 Create repository

Github 로그인





Github 자격 증명 관리(SSH키 등록)

실습1

(1) 본인PC에서 아래를 참조하여 SSH키 생성 및 SSH키 페어 환경 만들기

MINGW64:/c/Users/master

```
master@DESKTOP-S3CV8RN MINGW64 ~
$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/c/Users/master/.ssh/id_rsa):
/c/Users/master/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/master/.ssh/id_rsa
Your public key has been saved in /c/Users/master/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:byvl5LDlbSOgvQMvpXTw2xgYUg7KUWpX2tvqF4kHrpk master@DESKTOP-S3CV8RN
The key's randomart image is:
+---[RSA 3072]----+
|   .. .          |
|  ...+.          |
| .ooo+.          |
| .o.. =o         |
|     o.BS.       |
|      *.@.+      |
|     =.X ^o.     |
|    E.+ X.*.+    |
|      .o.+.o .   |
+----[SHA256]-----+
```

```
$ ls -al ~/.ssh/id_rsa*
-rw-r--r-- 1 master 197121 2610 Nov 21 15:01 /c/Users/master/.ssh/id_rsa
-rw-r--r-- 1 master 197121  576 Nov 21 15:01 /c/Users/master/.ssh/id_rsa.pub
```

개인PC 환경 구성

- 작업위치 : 홈폴더/.ssh
- 개인키 이름 변경: mv id_rsa github-id_rsa
- 환경파일작성 : config

```
Host github.com
  Hostname github.com
  IdentityFile ~/.ssh/github-id_rsa
```

id_rsa : 개인키로 적당한 이름으로 바꾸어 로컬에 보관
id_rsa.pub : 공개키로 github.com 사이트에 배치

Github 자격 증명 관리(SSH키 등록)

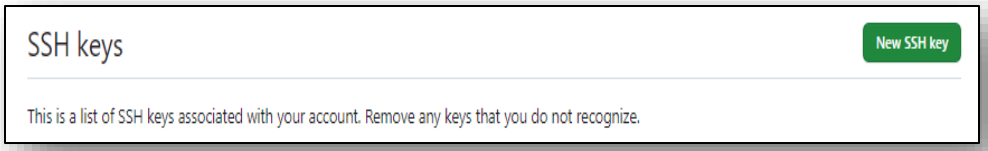
실습1

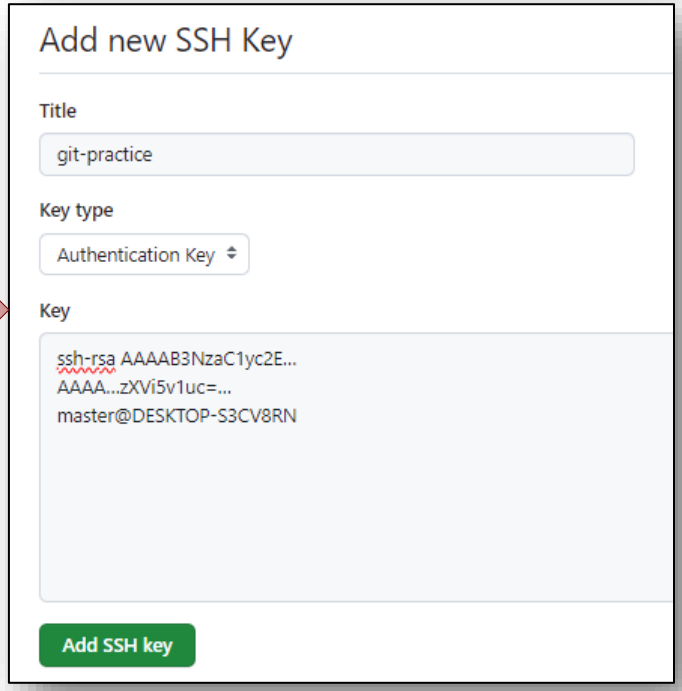
(1) 본인ID로 Github에 로그인한 후 앞에서 생성한 SSH공개키를 등록하기

$ cat id_rsa.pub

```
ssh-rsa AAAAB3NzaC1yc2E…
AAAA…zXVi5v1uc=…
master@DESKTOP-S3CV8RN
```

이 파일을 복사하여
github.com > Settings > SSH and GPG Keys

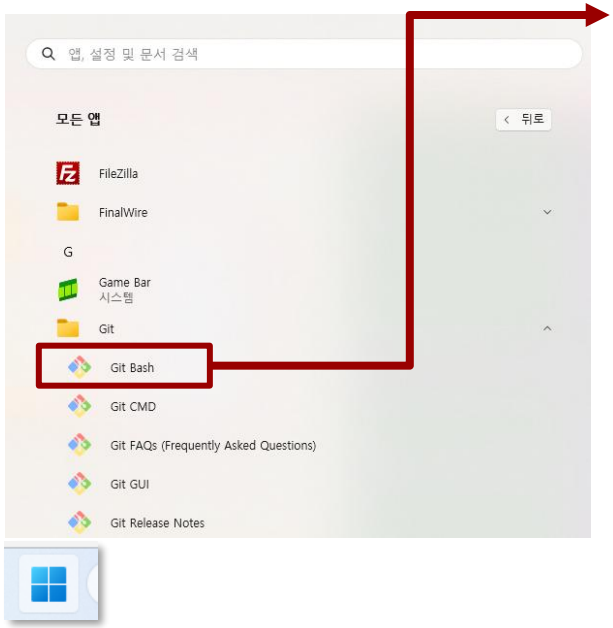




Github 자격 증명 관리(SSH키 활용)

실습1-로컬

(1) 앞에서 클론으로 생성한 로컬저장소(git-practice)에서 아래 내용을 참조하여 실행하기




```
master@DESKTOP-S3CV8RN MINGW64 ~
$ pwd
/c/Users/master

master@DESKTOP-S3CV8RN MINGW64 ~
$ git -v
git version 2.42.0.windows.2

master@DESKTOP-S3CV8RN MINGW64 ~
$ cd ~/Documents/git-practice

master@DESKTOP-S3CV8RN MINGW64 ~/Documents/git-practice (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

master@DESKTOP-S3CV8RN MINGW64 ~/Documents/git-practice (main)
$ git remote -v
origin  https://github.com/[ User-ID ]/git-practice.git (fetch)
origin  https://github.com/[ User-ID ]/git-practice.git (push)
master@DESKTOP-S3CV8RN MINGW64 ~/Documents/git-practice (main)
$ git remote add ssh-origin git@github.com:[ User-ID ]/git-practice.git

master@DESKTOP-S3CV8RN MINGW64 ~/Documents/git-practice (main)
$ git remote -v
origin  https://github.com/yoojaemyeong/git-practice.git (fetch)
origin  https://github.com/yoojaemyeong/git-practice.git (push)
ssh-origin      git@github.com:[ User-ID ]/git-practice.git (fetch)
ssh-origin      git@github.com:[ User-ID ]/git-practice.git (push)
```

원격저장소 (웹)

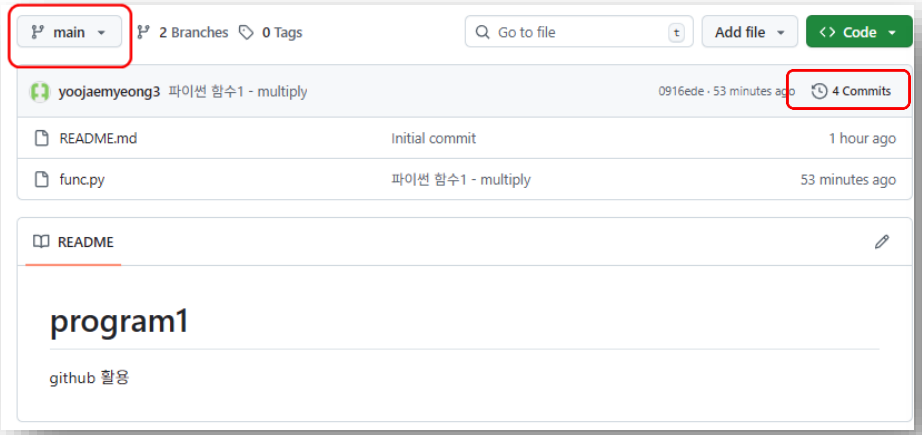
SSH 원격저장소 추가

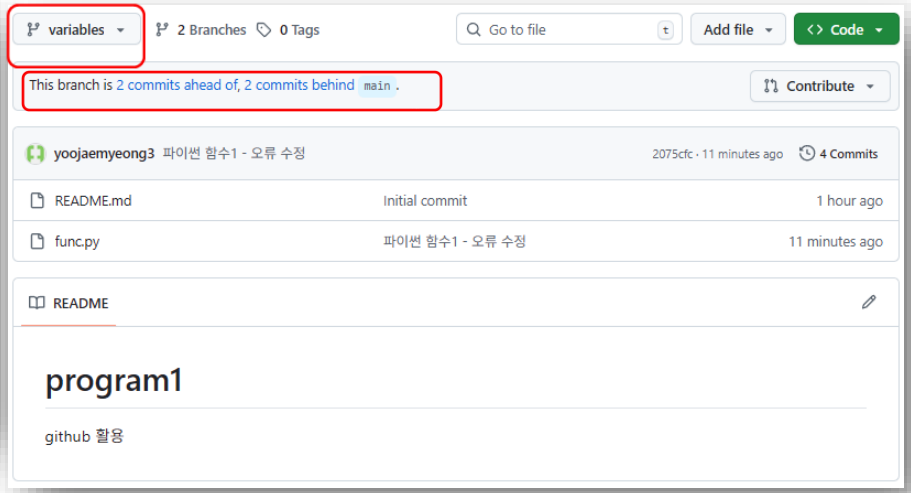
원격저장소(SSH) 추가 상태

Github에서 버전관리 실습2

1. New repository : python1
   - 이름: python1
   - 설명: 파이썬 버전관리
   - README.md 생성

2. 파일 생성
   - 파일명: func.py 파일만으로 버전 관리
   - main: 1st (add) > 2nd (sub) > 3rd (multiply) 커밋 상태 확인
   - variables: 1st 상태에서 브릿지 생성 > add_all(*args)





Github에서 버전관리 실습2

Commits

main

All users

All time

Commits on Sep 3, 2025

파이썬 함수1 - multiply yoojaemyeong3 authored 1 hour ago	Verified	0916ede
파이썬 함수1 - sub yoojaemyeong3 authored 1 hour ago	Verified	aa50f81
파이썬 함수1 - add yoojaemyeong3 authored 1 hour ago	Verified	288de10
Initial commit yoojaemyeong3 authored 1 hour ago	Verified	4920d7b

Files

aa50f81

Switch branches/tags

Find or create a branch...

Branches Tags

main default

variables

View all branches

program1 / func.py

yoojaemyeong3 파이썬 함수1 - sub

Code Blame 10 lines (9 loc) · 188 Bytes

```
def add(a,b):
    return a+b
def sub(a,b):
    return a-b

if __name__=="__main__":
    a = 3
    b = 5
    print(f"{a} + {b} = {add(a,b)}")
    print(f"{a} - {b} = {sub(a,b)}")
```

각 커밋 상태의 파일 내용
확인 및
이 상태에서 브런치를 생성할 수도 있음

Github에서 버전관리 실습3

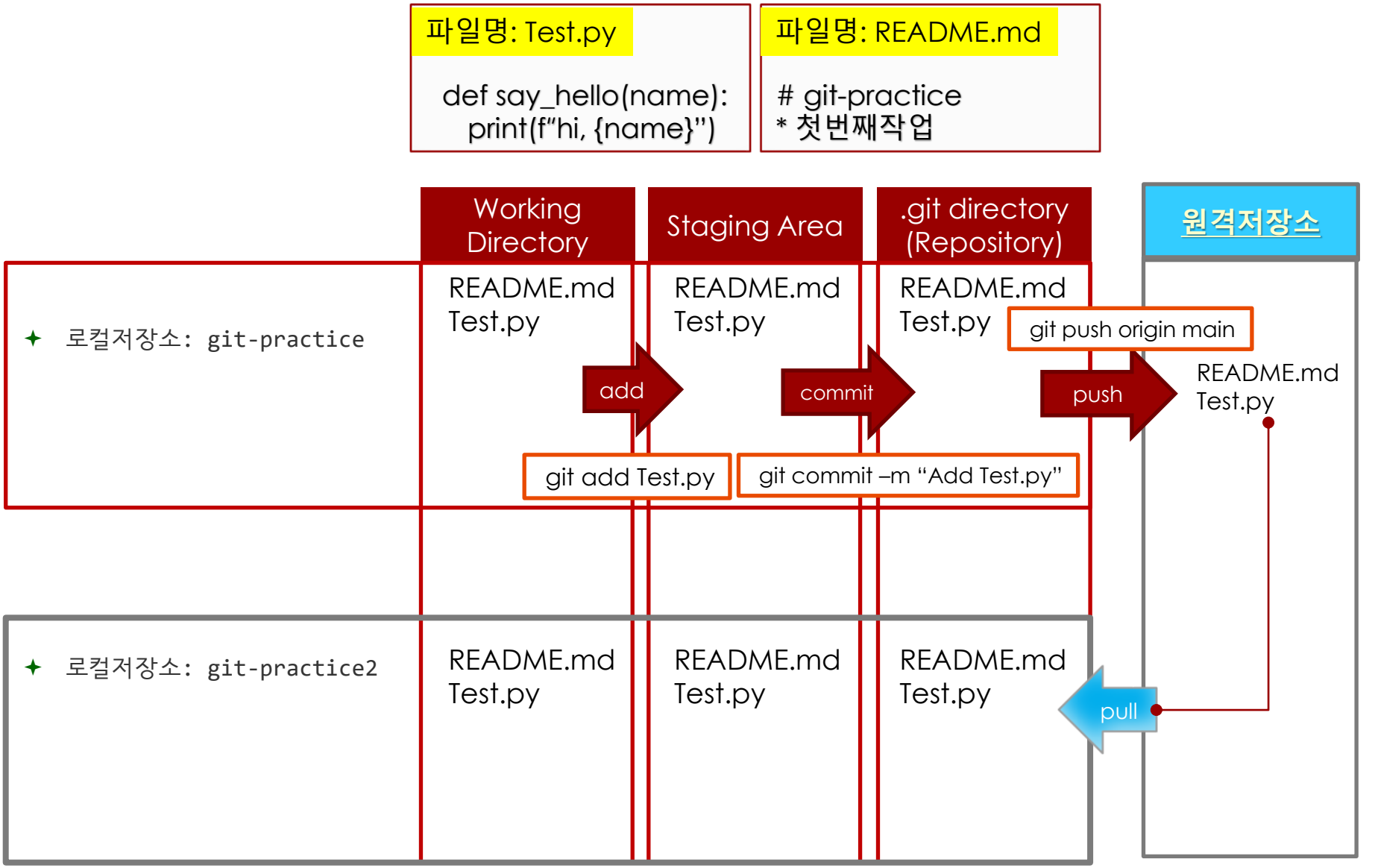
✦ [실습3]

- 실습환경 구축에서 생성한 로컬저장소(mywork)를 원격저장소로 push 하기

1. 원격저장소 만들기 : mywork
   - Github에서 빈 Repository 만들기
2. [로컬PC] 원격저장소 별칭 만들기
   - `git remote add` [원격저장소별칭] [원격저장소URL]
   - 예시: `git remote add origin https://github.com/[userid]/mywork.git`
3. [로컬PC] 원격저장소 확인
   - `git remote -v`
4. [로컬PC] to 원격저장소 동기화 하기
   - `git push` [원격저장소별칭] [로컬브런치]

```
git push origin main
Enumerating objects: 46, done.
Counting objects: 100% (46/46), done.
Delta compression using up to 24 threads
Compressing objects: 100% (33/33), done.
Writing objects: 100% (46/46), 18.36 KiB | 3.67 MiB/s, done.
Total 46 (delta 15), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (15/15), done.
To https://github.com/yoojaemyeong/mywork.git
 * [new branch]      main -> main
```

전체 흐름


파일명: Test.py
def say_hello(name):
print(f"hi, {name}")
파일명: README.md
git-practice
* 첫번째작업
Working Directory
Staging Area
.git directory (Repository)
원격저장소
✦ 로컬저장소: git-practice
README.md
Test.py
add
commit
push
git push origin main
git add Test.py
git commit –m "Add Test.py"
✦ 로컬저장소: git-practice2
pull

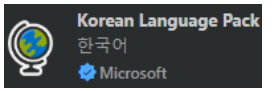
3. Visual Code 연계

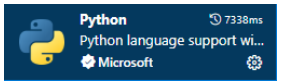
Visual Code 설치

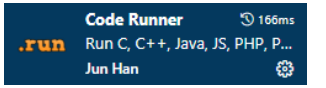
✦ 선수조건

- Git 설치(https://git-scm.com/downloads)
- 파이썬 설치(https://python.org 최신버전)

✦ VS code 설치

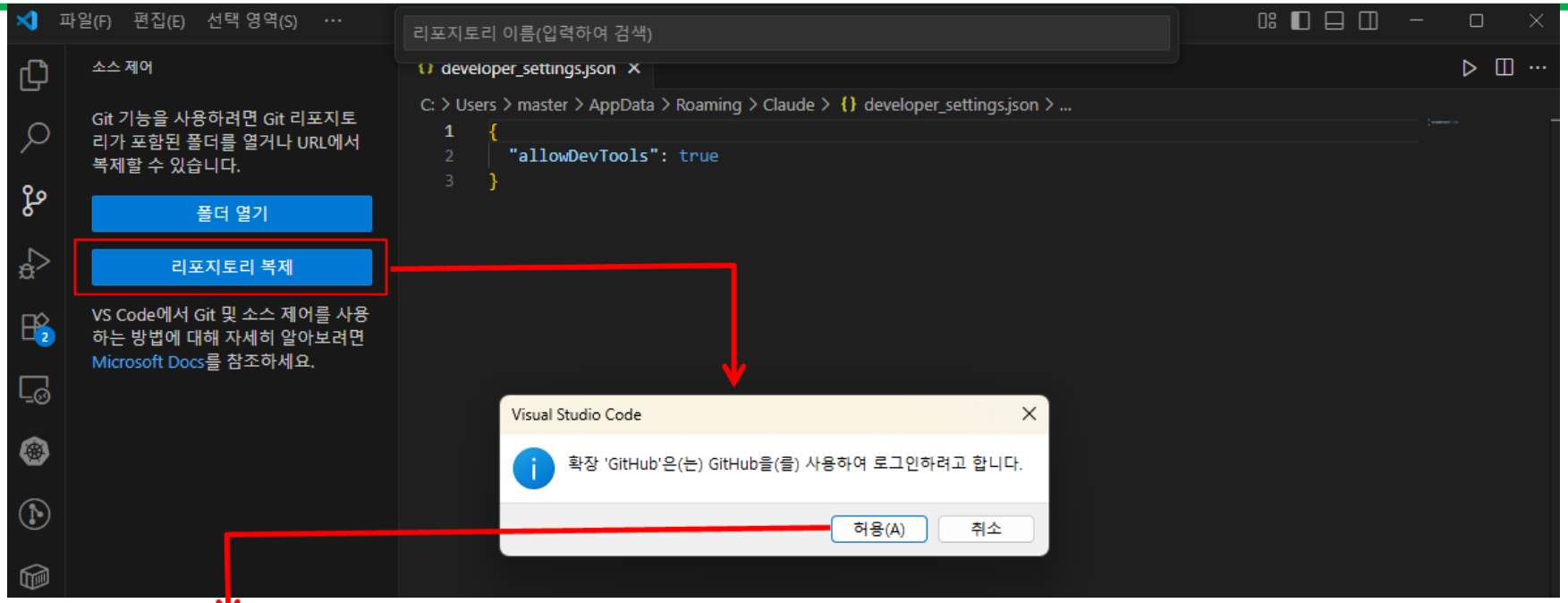
- https://code.visualstudio.com 다운로드 및 설치
- 필수 확장
  - 검색창: korean 한국어 언어팩 설치 

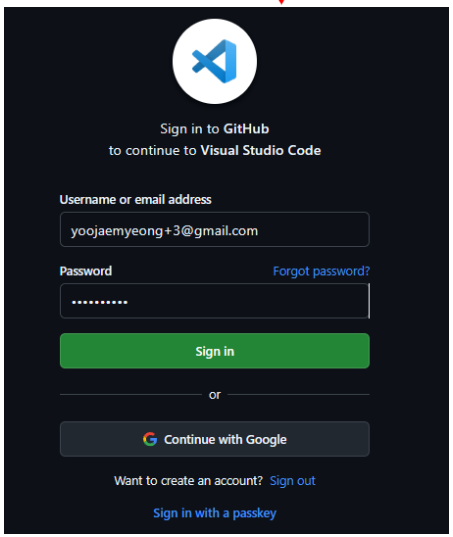
  - 검색창: Python 설치 

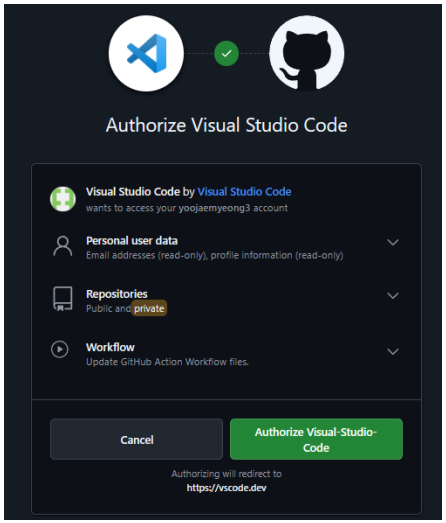
  - 검색창: code runner 단축키를 사용해 코드를 쉽고 빠르게 실행 

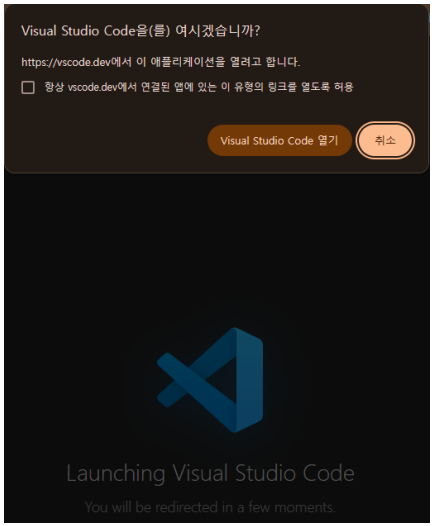
  - VS Code는 Git 내장되어 있음

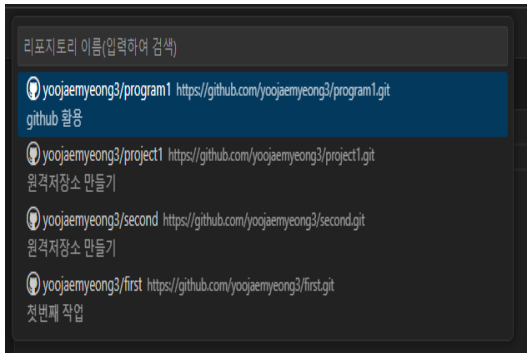
Visual Code에서 원격저장소 가져오기



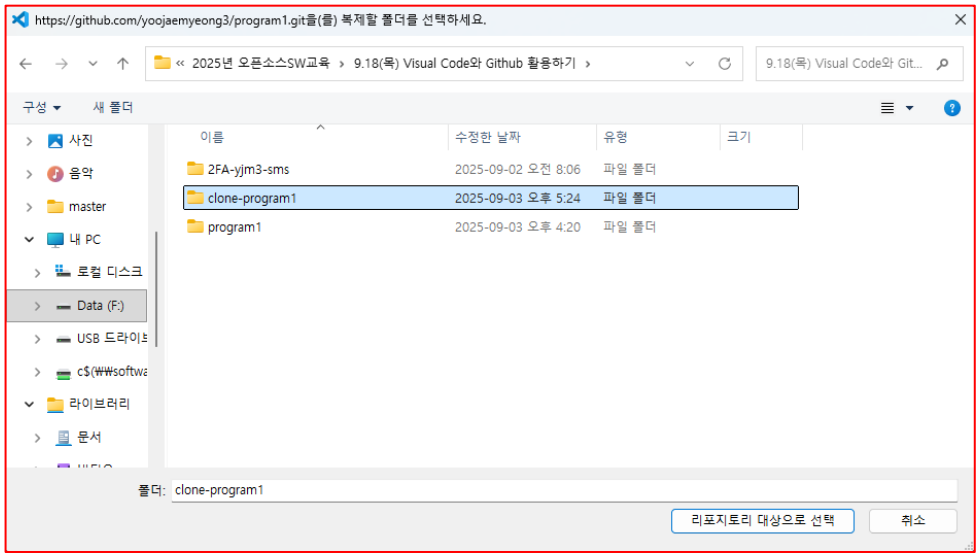


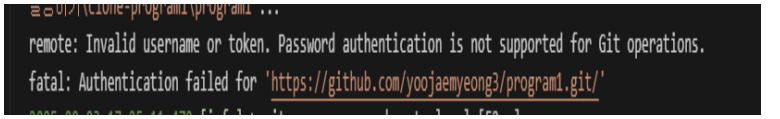


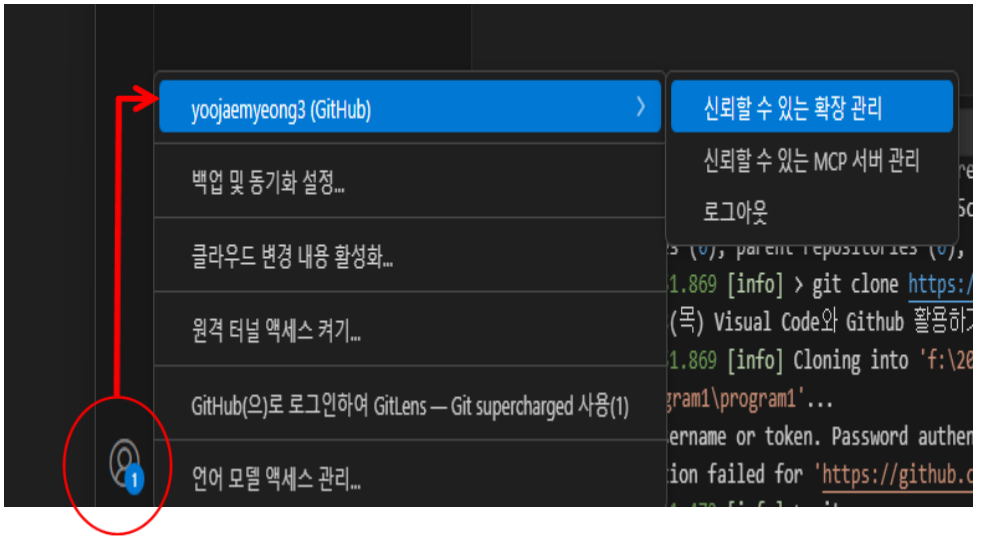


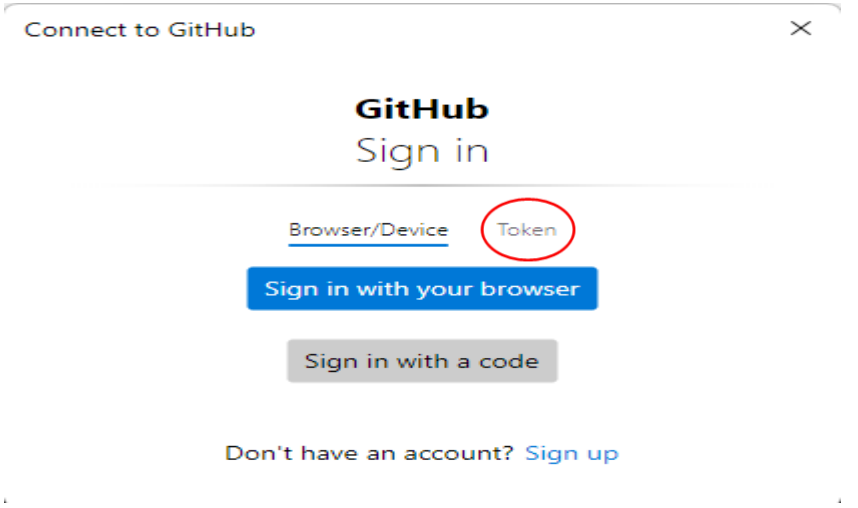


복제 에러 발생


https://github.com/yoojaemyeong3/program1.git을(를) 복제할 폴더를 선택하세요.
« 2025년 오픈소스SW교육 › 9.18(목) Visual Code와 Github 활용하기 ›
9.18(목) Visual Code와 Git...
구성
새 폴더
사진
음악
master
내 PC
로컬 디스크
Data (F:)
USB 드라이브
c$(₩₩softwa
라이브러리
문서
이름
수정한 날짜
유형
크기
2FA-yjm3-sms
2025-09-02 오전 8:06
파일 폴더
clone-program1
2025-09-03 오후 5:24
파일 폴더
program1
2025-09-03 오후 4:20
파일 폴더
폴더: clone-program1
리포지토리 대상으로 선택
취소


remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/yoojaemyeong3/program1.git/'


yoojaemyeong3 (GitHub)
백업 및 동기화 설정...
클라우드 변경 내용 활성화...
원격 터널 액세스 켜기...
GitHub(으)로 로그인하여 GitLens — Git supercharged 사용(1)
언어 모델 액세스 관리...
신뢰할 수 있는 확장 관리
신뢰할 수 있는 MCP 서버 관리
로그아웃


Connect to GitHub
GitHub
Sign in
Browser/Device
Token
Sign in with your browser
Sign in with a code
Don't have an account? Sign up

Visual Code에서 Github 저정소 복제

파일(F) 편집(E) 선택 영역(S) 보기(V) 이동(G) 실행(R) 터미널(T) 도움말(H) program1-1

소스 제어

변경 내용

메시지(Ctrl+Enter(으)로 "mai...

✓ 커밋

그래프 자동

파이썬 함수1 - multiply... main

파이썬 함수1 - sub yoojaemyeong3

파이썬 함수1 - add yoojaemyeong3

Initial commit yoojaemyeong3

func.py

func.py > add

yoojaemyeong3, 2시간 전 | 1 author (yoojaemyeong3)

```python
def add(a,b):
    return a+b
def sub(a,b):
    return a-b
def multiply(a,b):
    return a*b

if __name__=="__main__":
    a = 3
    b = 5
    print(f"{a} + {b} = {add(a,b)}")
    print(f"{a} - {b} = {sub(a,b)}")
    print(f"{a} * {b} = {multiply(a,b)}")
```

yoojaemyeong3, 2시간 전 • 파이썬 함수1 - add ...

Visual Code 작업 완료

파일(F) 편집(E) 선택 영역(S) 보기(V) 이동(G) 실행(R) 터미널(T) 도움말(H) program1-1

소스 제어

변경 내용

파이썬 함수1 : devide 추가

✓ 커밋

변경 사항 1

func.py M

그래프 자동

파이썬 함수1 - multiply... main

파이썬 함수1 - sub yoojaemyeo...

파이썬 함수1 - add yoojaemyeong3

Initial commit yoojaemyeong3

func.py M

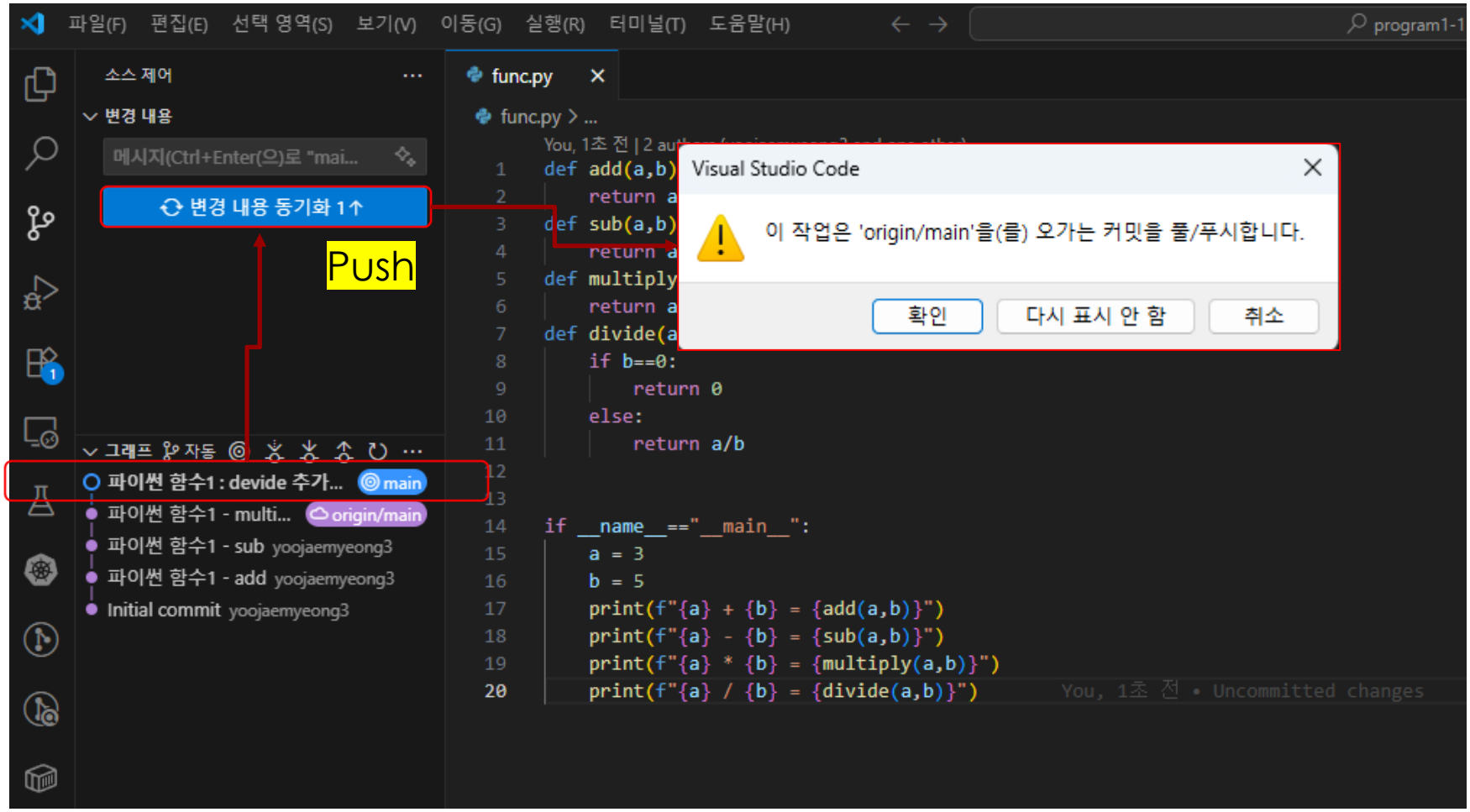
func.py > ...

You, 1초 전 | 2 authors (yoojaemyeong3 and one other)

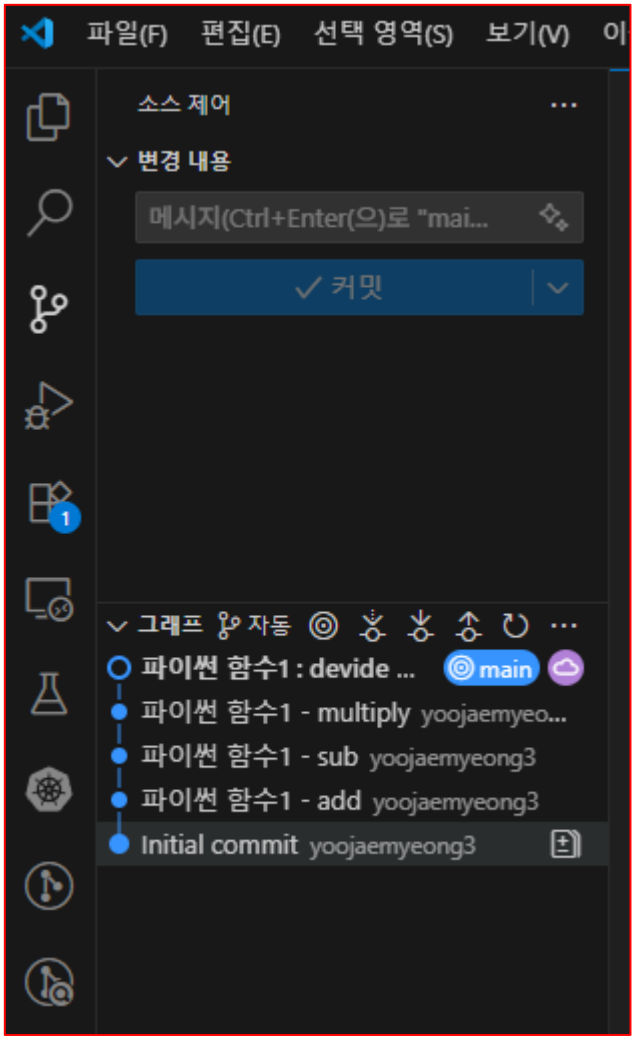
```python
def add(a,b):
    return a+b
def sub(a,b):
    return a-b
def multiply(a,b):
    return a*b
def divide(a,b):
    if b==0:
        return 0
    else:
        return a/b


if __name__=="__main__":
    a = 3
    b = 5
    print(f"{a} + {b} = {add(a,b)}")
    print(f"{a} - {b} = {sub(a,b)}")
    print(f"{a} * {b} = {multiply(a,b)}")
    print(f"{a} / {b} = {divide(a,b)}")
```

You, 1초 전 • Uncommitted changes

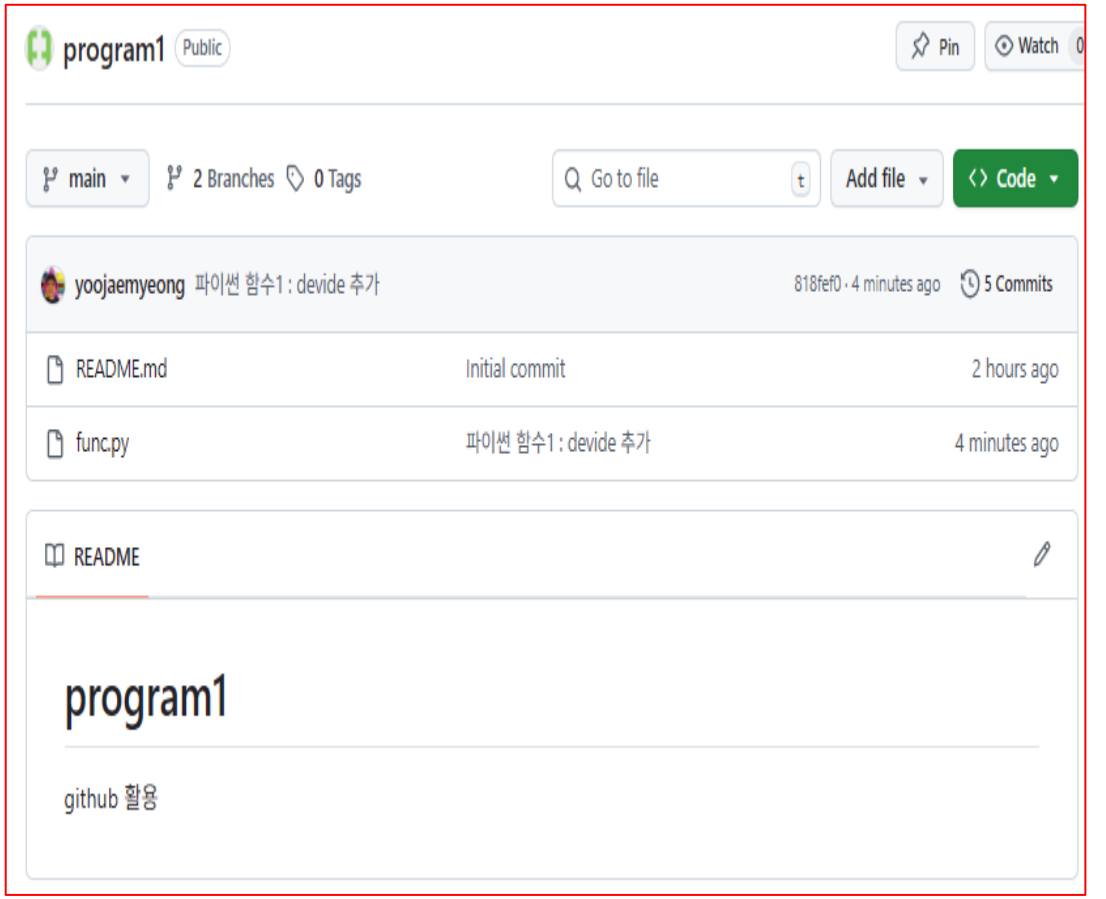
Visual Code 작업 완료 후 commit



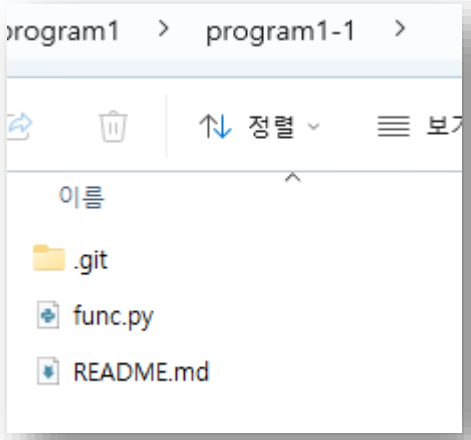
Visual Code, Github 동기화



=



로컬PC에서 git history 보기



로그보기 명령어

git log --oneline --all --decorate --graph

```
PS F:\2025년 오픈소스SW교육\9.18(목) Visual Code와 Github 활용하기\clone-program1\program1-1> git log --oneline --all --decorate --graph
* 818fef0 (HEAD -> main, origin/main, origin/HEAD) 파이썬 함수1 : devide 추가
* 0916ede 파이썬 함수1 - multiply
* aa50f81 파이썬 함수1 - sub
| * 2075cfc (origin/variables) 파이썬 함수1 - 오류 수정
| * 08f4d9f 파이썬 함수1 - 가변인수처리 add_all
|/
* 288de10 파이썬 함수1 - add
* 4920d7b Initial commit
```

Github 활용

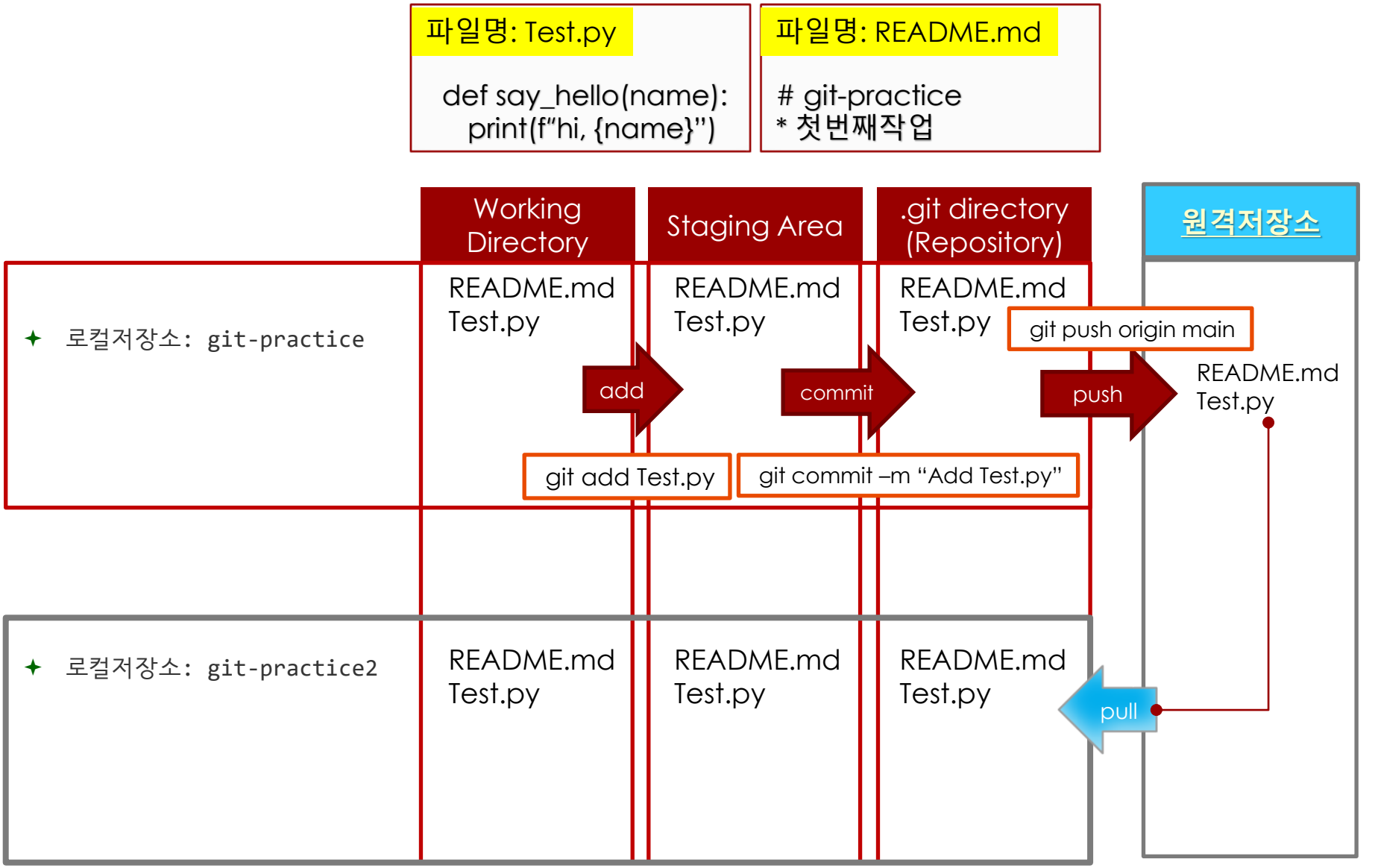
✦ [실습2]

- 실습환경 구축에서 생성한 로컬저장소(mywork)를 원격저장소로 push 하기

1. 원격저장소 만들기 : mywork
   - Github에서 빈 Repository 만들기
2. [로컬PC] 원격저장소 별칭 만들기
   - `git remote add` [원격저장소별칭] [원격저장소URL]
   - 예시: `git remote add origin https://github.com/[userid]/mywork.git`
3. [로컬PC] 원격저장소 확인
   - `git remote -v`
4. [로컬PC] to 원격저장소 동기화 하기
   - `git push` [원격저장소별칭] [로컬브런치]

```
git push origin main
Enumerating objects: 46, done.
Counting objects: 100% (46/46), done.
Delta compression using up to 24 threads
Compressing objects: 100% (33/33), done.
Writing objects: 100% (46/46), 18.36 KiB | 3.67 MiB/s, done.
Total 46 (delta 15), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (15/15), done.
To https://github.com/yoojaemyeong/mywork.git
 * [new branch]      main -> main
```

전체 흐름


파일명: Test.py
def say_hello(name):
print(f"hi, {name}")
파일명: README.md
git-practice
* 첫번째작업
Working Directory
Staging Area
.git directory (Repository)
원격저장소
✦ 로컬저장소: git-practice
README.md
Test.py
add
commit
push
git push origin main
git add Test.py
git commit –m "Add Test.py"
✦ 로컬저장소: git-practice2
pull

4. 구글 클라우드 + colab

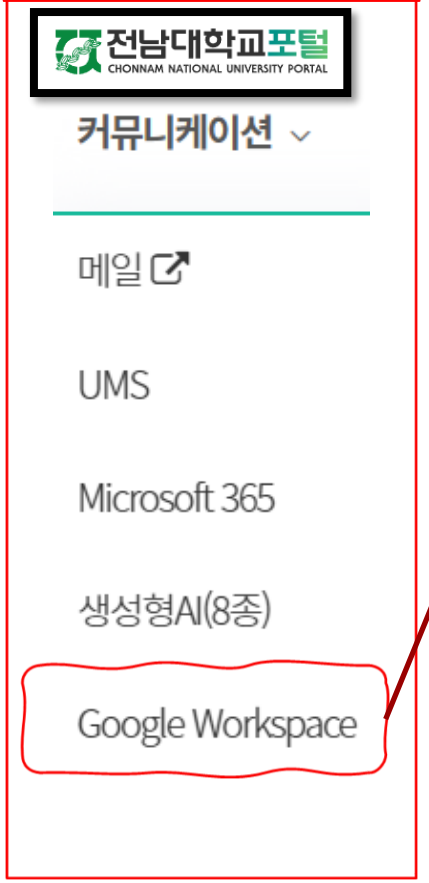
작업 순서

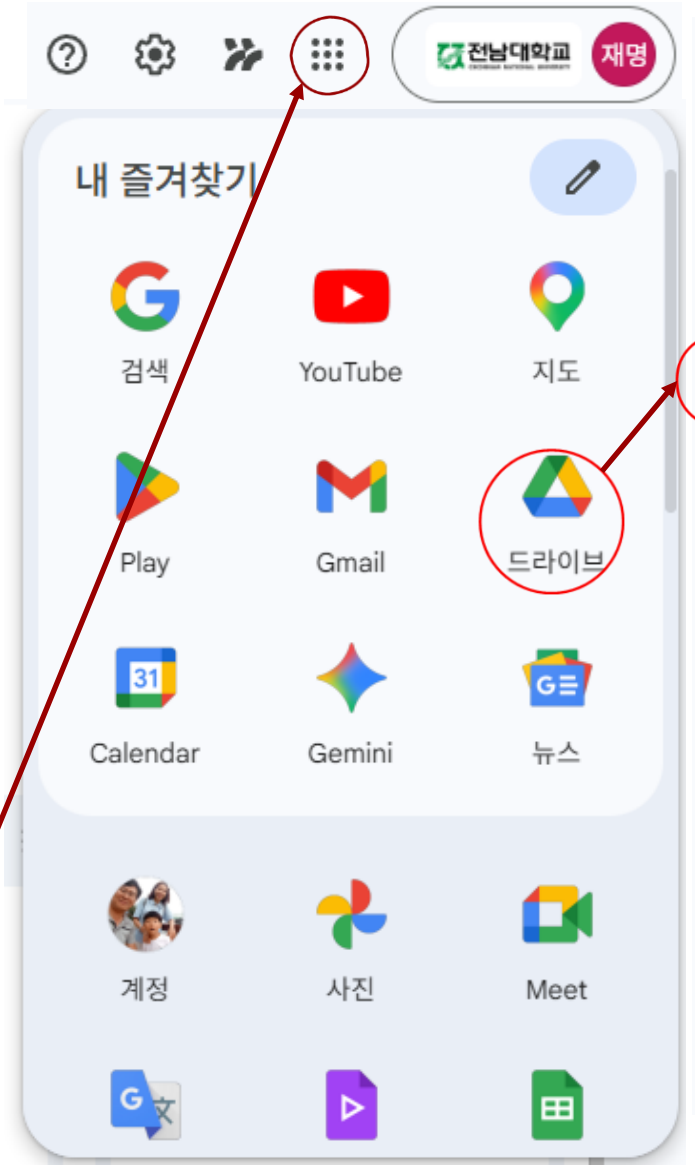
전남대포털 로그인 → 커뮤니케이션 Google Workspace → Github 원격저장소 생성 → 구글 드라이브 => Colab 작업

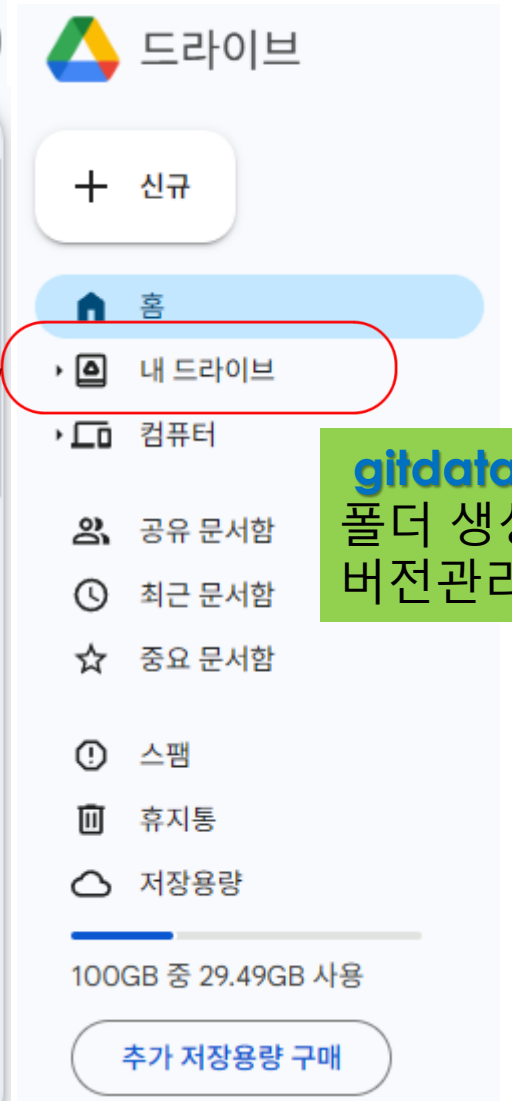
구글 클라우드 / colab 활용

구글 로그인

구글 전남대 계정



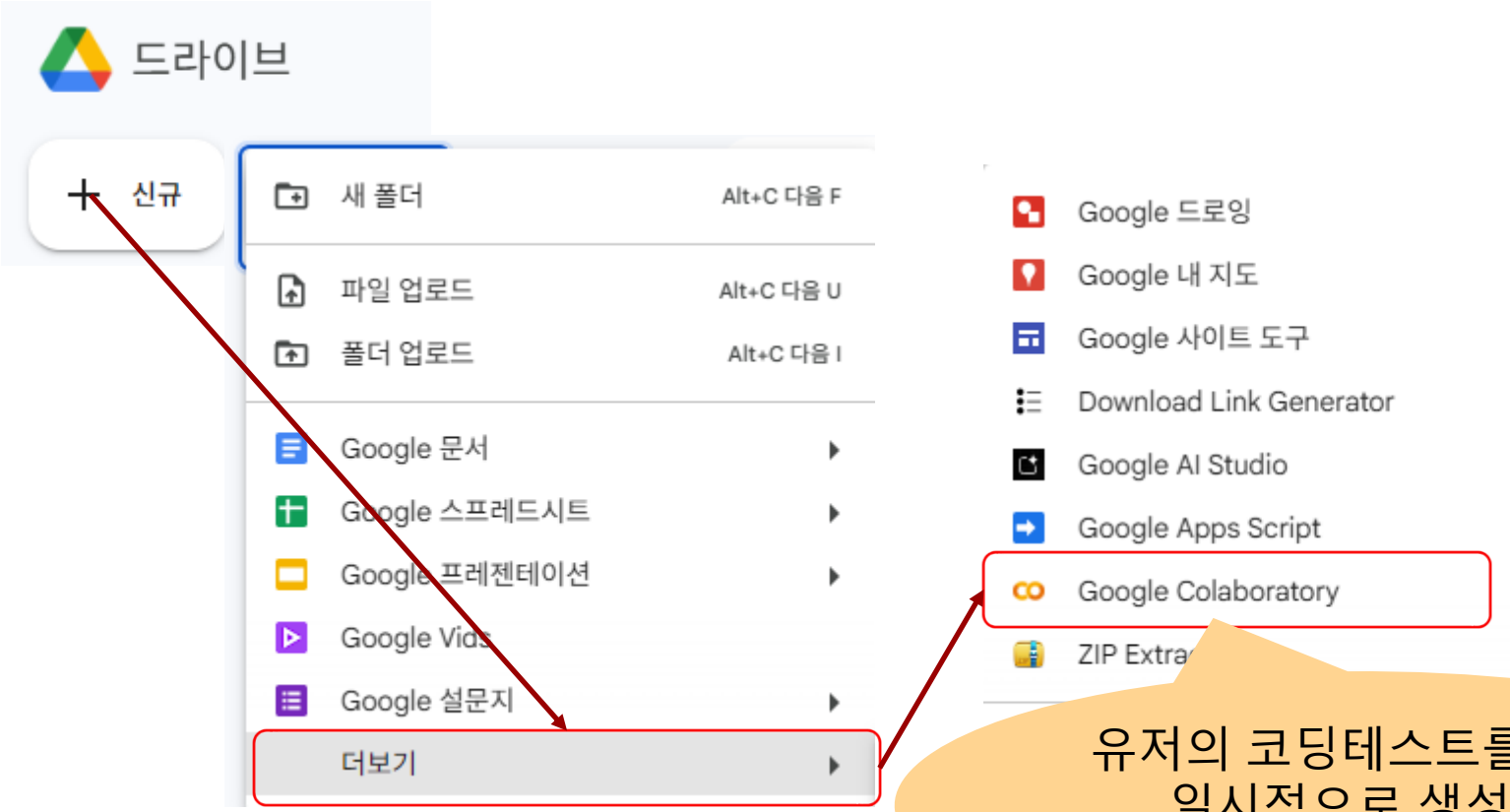




gitdata
폴더 생성하여
버전관리에 활용

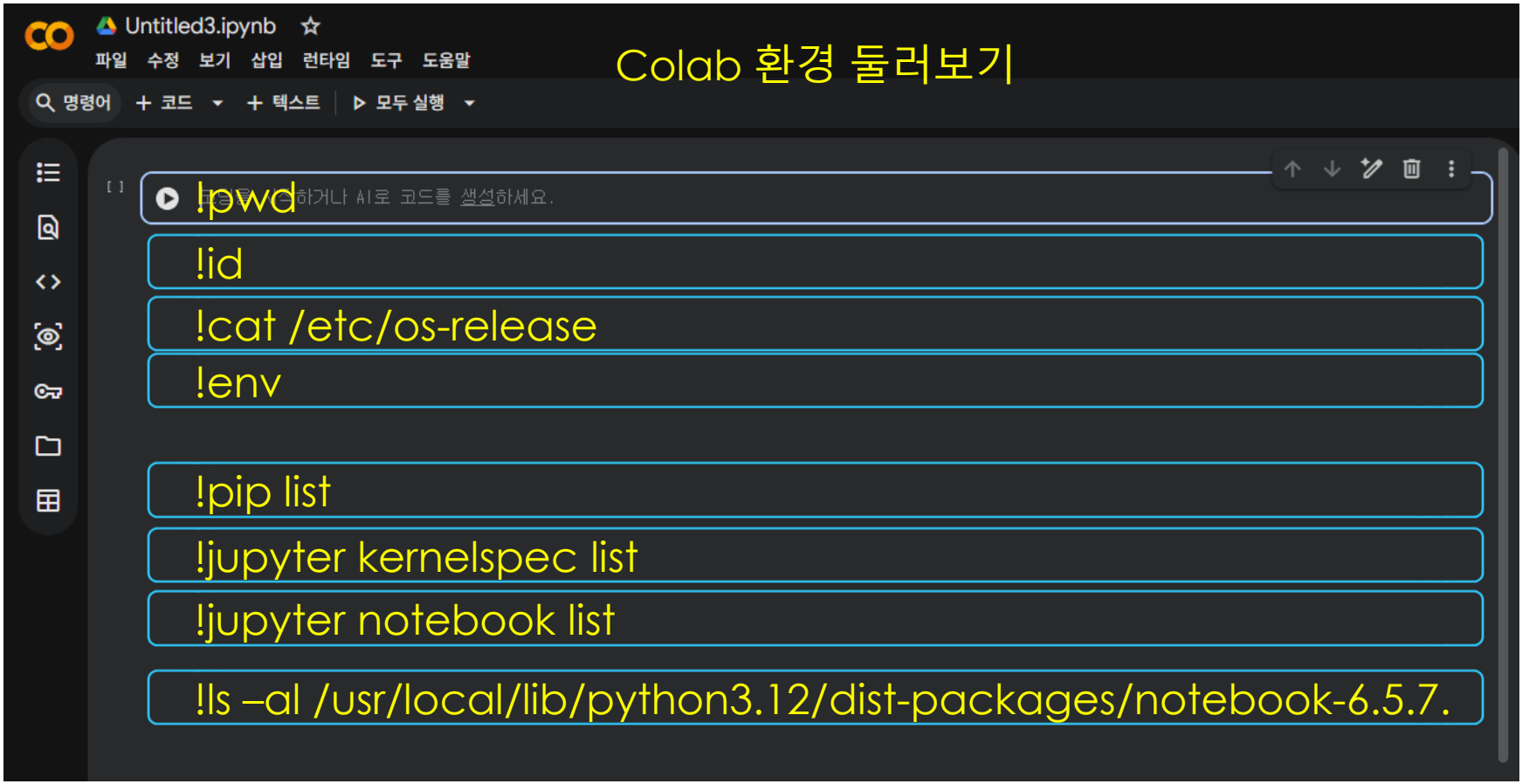
구글 클라우드 / colab 활용

✦ 구글 드라이브에서 colab 활용



유저의 코딩테스트를 위해
일시적으로 생성된
리눅스(Ubuntu22.04) 컨테이너

구글 클라우드 / colab 활용



구글 클라우드 / Colab 활용

1단계: Google Drive 마운트

```
from google.colab import drive
drive.mount('/content/g')
```

```
Mounted at /content/g
```

2단계: Github repository clone(gDrive)

```
[ ]
!git clone https://github.com/yoojaemyeong/gDrive.git
```

```
Cloning into 'gDrive'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

3단계: Drive 폴더로 이동 (gDrive --> 구글 드라이브로 이동)

```
!mv gDrive /content/g/
```

```
!ls -al /content/g/MyDrive/gitdata/
```

```
%cd /content/g/MyDrive/gitdata/
```

구글 클라우드 / Colab 활용

✦ 새 파일 작업

- 직접 만들기 또는 구글 드라이브에 업로드

```
[ ]
%%writefile hello.py
print("안녕하세요")

Writing hello.py
```

✦ 파일 추가 및 커밋

```
!git add hello.py

!git commit -m "Add hello.py"

On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

!git log --oneline  --all --decorate --graph

* b8d9166 (HEAD -> main) Add hello.py
* f63eb2a (origin/main, origin/HEAD) Initial commit
```

✦ 원격저장소 상태

```
[ ]
!git remote -v

origin  https://github.com/yoojaemyeong/gDrive.git (fetch)
origin  https://github.com/yoojaemyeong/gDrive.git (push)

[ ]
!git config --global --list

user.name=Yoo Jae Myeong
user.email=yoojaemyeong@google.com
```

구글 클라우드 / Colab 활용

✦ 원격저장소 인증코드 등록 및 push(버전동기화)

```
[ ]
!git remote set-url origin https://jmyoo-mytoken:ghp_eRfETd[redacted]uFLkW@github.com/yoojaemyeong/gDrive.git

[ ]
!git push origin main

Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 312 bytes | 28.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/yoojaemyeong/gDrive.git
   f63eb2a..b8d9166  main -> main
```

감사합니다.